Mastering Java 8

Functional Programming, Streams, and Enterprise Usage Guide

Lambda Expressions | Streams API | Functional Interfaces | CompletableFuture | Optional | Collectors

Enterprise Reference Guide & Interview Preparation

Version 1.0 | March 2026

Table of Contents

Part I – Foundations

Chapter 1: Introduction to Java 8

Chapter 2: Why Java 8 Was Introduced

Chapter 3: Functional Programming Concepts

Chapter 4: Java 8 Architecture and Core Changes

Part II – Core Features

Chapter 5: Lambda Expressions

Chapter 6: Functional Interfaces

Chapter 7: Method References

Part III – Streams & Data Processing

Chapter 8: Streams API – Fundamentals

Chapter 9: Stream Operations Deep Dive

Chapter 10: Collectors

Part IV – Advanced Features

Chapter 11: Optional Class

Chapter 12: CompletableFuture

Chapter 13: Parallel Streams

Chapter 14: Default & Static Methods in Interfaces

Part V – Interview Preparation

Chapter 15: Beginner & Intermediate Questions

Chapter 16: Advanced & Architect-Level Questions

Chapter 17: Coding Exercises

Chapter 1: Introduction to Java 8

Understanding the most transformative release in Java's history

1.1 What is Java 8?

Java 8, released on **March 18, 2014**, is widely considered the most significant update to the Java programming language since its inception. It introduced functional programming paradigms to a language that had been predominantly object-oriented for nearly two decades.

Java 8 brought a fundamental shift in how developers write Java code—moving from imperative, verbose patterns to declarative, concise, and expressive styles. It was not merely an incremental update; it was a paradigm shift that made Java competitive with modern languages like Scala, Kotlin, and Python in terms of expressiveness.

1.2 Key Features at a Glance

Feature	Category	Impact Level
Lambda Expressions	Language	Revolutionary
Functional Interfaces	API	High
Stream API	API	Revolutionary
Optional Class	API	High
Method References	Language	Medium
Default Methods	Language	High
CompletableFuture	API	High
New Date/Time API	API	High
Nashorn JavaScript Engine	Platform	Medium
Parallel Streams	API	Medium

1.3 Java 8 in the Enterprise Landscape

Java 8 became the backbone of modern enterprise Java development. Frameworks like Spring Boot, Hibernate, and Apache Kafka adopted Java 8 features extensively. Understanding Java 8 is essential for working with:

- **Spring Boot** – Uses lambdas in configurations, functional endpoints, and reactive programming
- **Hibernate / JPA** – Stream-based query results, Optional return types
- **Microservices** – CompletableFuture for async communication, Streams for data transformation
- **Apache Kafka** – Stream processing, functional serialization/deserialization
- **REST APIs** – Functional-style request/response processing, validation chains

Chapter 2: Why Java 8 Was Introduced

The motivations, industry pressures, and technical debt that drove this release

2.1 Industry Pressures

By 2014, Java was facing significant competitive pressure from modern languages:

- **Scala** had proven that functional programming on the JVM was viable and productive
- **Python** and **Ruby** offered far more concise syntax for common operations
- **C# / .NET** had already introduced LINQ with lambda expressions years earlier
- **Multi-core processors** were becoming standard, requiring better concurrency abstractions
- **Big Data** processing (Hadoop, Spark) demanded functional data transformation capabilities

2.2 Technical Problems with Pre-Java 8 Code

Problem 1: Verbose Anonymous Inner Classes

```
// Pre-Java 8: Sorting a list of strings by length
Collections.sort(names, new Comparator<String>() {
    @Override
    public int compare(String a, String b) {
        return Integer.compare(a.length(), b.length());
    }
});

// Java 8: Same operation in one line
names.sort(Comparator.comparingInt(String::length));
```

Problem 2: No Native Support for Parallel Processing

```
// Pre-Java 8: Manual thread management for parallel processing
ExecutorService executor = Executors.newFixedThreadPool(4);
List<Future<Result>> futures = new ArrayList<>();
for (Order order : orders) {
    futures.add(executor.submit(() -> processOrder(order)));
}
// Manual collection of results...
List<Result> results = new ArrayList<>();
for (Future<Result> f : futures) {
    results.add(f.get()); // Blocking, exception handling needed
}
executor.shutdown();

// Java 8: Parallel stream
List<Result> results = orders.parallelStream()
    .map(this::processOrder)
    .collect(Collectors.toList());
```

Problem 3: Null Pointer Exceptions Everywhere

```
// Pre-Java 8: Defensive null checking
public String getCityName(User user) {
    if (user != null) {
        Address address = user.getAddress();
        if (address != null) {
            City city = address.getCity();
            if (city != null) {
                return city.getName();
            }
        }
    }
    return "Unknown";
}

// Java 8: Optional chaining
public String getCityName(User user) {
    return Optional.ofNullable(user)
        .map(User::getAddress)
        .map(Address::getCity)
        .map(City::getName)
        .orElse("Unknown");
}
```

Problem 4: Poor Date/Time API

```
// Pre-Java 8: Mutable, thread-unsafe java.util.Date
Date date = new Date();
date.setYear(2024 - 1900);  // Year offset from 1900!
date.setMonth(0);            // Month is 0-indexed!

// Java 8: Immutable, clear java.time API
LocalDate date = LocalDate.of(2024, Month.JANUARY, 1);
```

Problem 5: Inability to Evolve Interfaces

```
// Pre-Java 8: Adding a method to interface broke all implementations
public interface Collection<E> {
    // If you add stream() here, EVERY class implementing
    // Collection must be updated — not feasible!
}

// Java 8: Default methods solve this
public interface Collection<E> {
    default Stream<E> stream() {
        return StreamSupport.stream(spliterator(), false);
    }
}
```

💡 **Key Insight**

Java 8 wasn't just about adding new features—it was about solving fundamental design limitations that had accumulated over 18 years of Java's evolution. Every feature was carefully designed to address specific pain points in enterprise development.

Chapter 3: Functional Programming Concepts

Core principles of functional programming that underpin Java 8's design

3.1 What is Functional Programming?

Functional Programming (FP) is a programming paradigm where programs are constructed by applying and composing **functions**. It treats computation as the evaluation of mathematical functions and avoids changing state and mutable data.

Core Principles

Principle	Description	Java 8 Feature
First-Class Functions	Functions can be passed as arguments, returned from other functions, and assigned to variables	Lambda expressions, Method references
Pure Functions	Functions that always produce the same output for the same input, with no side effects	Streams (encourage stateless operations)
Immutability	Data should not be modified after creation	Stream operations produce new streams, not modify existing ones
Higher-Order Functions	Functions that take other functions as parameters or return them	Predicate, Function, Consumer, Supplier
Declarative Style	Describe WHAT to do, not HOW to do it	Stream API pipeline operations
Composition	Combine simple functions to build complex ones	Function.andThen(), Predicate.and()

3.2 Imperative vs Declarative Programming

```
// IMPERATIVE: How to do it (step by step instructions)
List<String> expensiveProductNames = new ArrayList<>();
for (Product product : products) {
    if (product.getPrice() > 100) {
        expensiveProductNames.add(product.getName().toUpperCase());
    }
}
Collections.sort(expensiveProductNames);

// DECLARATIVE: What to do (describe the result you want)
List<String> expensiveProductNames = products.stream()
    .filter(p -> p.getPrice() > 100)
    .map(p -> p.getName().toUpperCase())
    .sorted()
    .collect(Collectors.toList());
```

3.3 First-Class Functions in Java 8

In Java 8, functions became first-class citizens through **functional interfaces** and **lambda expressions**. This means functions can be:

```
// 1. Assigned to a variable
Function<String, Integer> toLength = String::length;

// 2. Passed as a method argument
List<String> sorted = names.stream()
    .sorted(Comparator.comparing(toLength))
    .collect(Collectors.toList());

// 3. Returned from a method
public Predicate<Employee> createSalaryFilter(double minSalary) {
    return emp -> emp.getSalary() > minSalary;
}

// 4. Stored in data structures
Map<String, Function<Double, Double>> operations = new HashMap<>();
operations.put("double", x -> x * 2);
operations.put("square", x -> x * x);
operations.put("negate", x -> -x);
```

3.4 Pure Functions and Side Effects

```
// PURE FUNCTION: No side effects, deterministic
Function<Integer, Integer> square = x -> x * x;
// square.apply(5) always returns 25

// IMPURE FUNCTION: Has side effects
int counter = 0;
Function<String, String> impure = s -> {
    counter++;  // Side effect: modifying external state
    System.out.println(s);  // Side effect: I/O
    return s.toUpperCase();
};
```

📘 **Enterprise Insight**

In enterprise systems, pure functions are critical in **stream pipelines**, **map-reduce operations**, and **parallel processing**. Side effects in parallel streams can cause race conditions and data corruption. Always design stream operations to be stateless and side-effect free.

3.5 Function Composition

```
// Building complex operations from simple functions
Function<String, String> trim = String::trim;
Function<String, String> toLowerCase = String::toLowerCase;
Function<String, String> removeSpaces = s -> s.replaceAll("\\s+", "-");

// Compose: trim -> lowercase -> replace spaces
Function<String, String> slugify = trim
    .andThen(toLowerCase)
    .andThen(removeSpaces);

String slug = slugify.apply("  Hello World  "); // "hello-world"

// Predicate composition
Predicate<Employee> isActive = Employee::isActive;
Predicate<Employee> isSenior = e -> e.getYearsOfExperience() > 5;
Predicate<Employee> earnsAbove100K = e -> e.getSalary() > 100000;

// Combine predicates
Predicate<Employee> eligibleForPromotion = isActive
    .and(isSenior)
    .and(earnsAbove100K.negate()); // Active seniors not yet earning 100K+
```

Chapter 4: Java 8 Architecture and Core Changes

Internal changes to the JVM, compiler, and standard library

4.1 JVM Changes

Java 8 introduced several JVM-level changes to support functional programming:

- **invokedynamic Bytecode** – Lambda expressions compile to `invokedynamic` instructions, not anonymous inner classes. This is more efficient in terms of memory and startup time.
- **PermGen Removal** – The permanent generation memory space was replaced with **Metaspace**, which uses native memory and grows dynamically.
- **Compact Profiles** – Subset profiles of the JRE for smaller deployment footprints.

4.2 Lambda vs Anonymous Inner Class – Under the Hood

```
Anonymous Inner Class Compilation: MyClass.java → MyClass.class + MyClass$1.class + MyClass$2.class ... (Each anonymous class = separate .class file = memory overhead) Lambda Expression Compilation: MyClass.java → MyClass.class (lambdas compiled as private methods) (invokedynamic links the lambda at runtime using LambdaMetafactory)
```

```
// How lambdas are compiled internally:
// 1. Lambda body becomes a private static/instance method
// 2. invokedynamic instruction generates implementation at first call
// 3. LambdaMetafactory creates the functional interface implementation

// This means:
// - No extra .class files (unlike anonymous inner classes)
// - No object creation on every call (can cache/reuse instances)
// - Better JIT optimization opportunities
// - Reduced memory footprint
```

4.3 Key Architectural Additions to java.util

Package	Purpose	Key Classes/Interfaces
java.util.function	Functional interfaces	Predicate, Function, Consumer, Supplier, BiFunction, etc.
java.util.stream	Stream processing	Stream, IntStream, LongStream, DoubleStream, Collectors
java.util.Optional	Null safety	Optional, OptionalInt, OptionalLong, OptionalDouble
java.time	Date/Time	LocalDate, LocalTime, LocalDateTime, ZonedDateTime, Duration, Period
java.util.concurrent	Async programming	CompletableFuture, CompletionStage

4.4 The Spliterator Architecture

The `Spliterator` (Splittable Iterator) is the underlying mechanism that enables both sequential and parallel stream processing:

```java
// Spliterator provides:
// 1. tryAdvance() - Process one element
// 2. trySplit()   - Split into two halves for parallel processing
// 3. estimateSize() - Estimate remaining elements
// 4. characteristics() - Properties like ORDERED, DISTINCT, SORTED

// Custom Spliterator example for batch processing
public class BatchSpliterator<T> implements Spliterator<List<T>> {
    private final Spliterator<T> source;
    private final int batchSize;

    @Override
    public boolean tryAdvance(Consumer<? super List<T>> action) {
        List<T> batch = new ArrayList<>(batchSize);
        while (batch.size() < batchSize && source.tryAdvance(batch::add));
        if (batch.isEmpty()) return false;
        action.accept(batch);
        return true;
    }

    @Override
    public Spliterator<List<T>> trySplit() {
        // Enable parallel batch processing
        Spliterator<T> split = source.trySplit();
        return split == null ? null : new BatchSpliterator<>(split, batchSize);
    }
    // ... estimateSize() and characteristics() implementations
}
```

🏗️ **Architecture Insight**

Understanding Spliterator is crucial for architect-level interviews. It explains WHY parallel streams work, HOW the ForkJoinPool splits work, and WHEN custom Spliterators are needed (e.g., processing database result sets in batches, reading large files in chunks).

Chapter 5: Lambda Expressions

The cornerstone of functional programming in Java

5.1 What are Lambda Expressions?

A lambda expression is an anonymous function—a block of code that can be passed around as a value. It provides a clear and concise way to implement single-method interfaces (functional interfaces). Lambdas enable you to treat functionality as a method argument, or code as data.

5.2 Lambda Syntax

```
// Full syntax
(Type1 param1, Type2 param2) → { statements; return value; }

// Simplified forms:
// 1. Single parameter - parentheses optional
name → name.toUpperCase()

// 2. Type inference - types can be omitted
(a, b) → a + b

// 3. Single expression - braces and return optional
x → x * x

// 4. No parameters
() → System.out.println("Hello")

// 5. Multiple statements - braces required
(x, y) → {
    int sum = x + y;
    System.out.println("Sum: " + sum);
    return sum;
}
```

5.3 Lambda Expression Examples

Example 1: Sorting

```
// Sort employees by salary descending, then by name ascending
List<Employee> sorted = employees.stream()
    .sorted(Comparator
        .comparingDouble(Employee::getSalary).reversed()
        .thenComparing(Employee::getName))
    .collect(Collectors.toList());

// Custom comparator with lambda
employees.sort((e1, e2) -> {
    int salaryCompare = Double.compare(e2.getSalary(), e1.getSalary());
    return salaryCompare != 0 ? salaryCompare : e1.getName().compareTo(e2.getName());
});
```

Example 2: Event Handling

```
// Pre-Java 8
button.addActionListener(new ActionListener() {
    @Override
    public void actionPerformed(ActionEvent e) {
        System.out.println("Button clicked!");
    }
});

// Java 8 Lambda
button.addActionListener(e -> System.out.println("Button clicked!"));
```

Example 3: Thread Creation

```
// Pre-Java 8
new Thread(new Runnable() {
    @Override
    public void run() {
        System.out.println("Running in thread: " + Thread.currentThread().getName());
    }
}).start();

// Java 8 Lambda
new Thread(() -> System.out.println("Running in thread: " + Thread.currentThread().ge
    .start();
```

5.4 Variable Capture and Closure

Lambdas can capture variables from their enclosing scope. However, captured variables must be **effectively final**—they cannot be modified after initialization.

```
// ✅ Valid: Effectively final variable
String prefix = "Hello";
Function<String, String> greeter = name -> prefix + " " + name;

// ❌ Invalid: Variable is modified
String prefix = "Hello";
prefix = "Hi";  // Now it's not effectively final
Function<String, String> greeter = name -> prefix + " " + name; // COMPILE ERROR

// ✅ Workaround: Use an array or AtomicReference for mutable state
AtomicInteger counter = new AtomicInteger(0);
List<String> numbered = names.stream()
    .map(name -> counter.incrementAndGet() + ". " + name)
    .collect(Collectors.toList());
```

> ⚠️ **Why Effectively Final?**
>
> Lambda expressions may be executed in a different thread (e.g., with parallel streams or CompletableFuture). If the captured variable could be modified, it would create race conditions. The "effectively final" rule prevents this class of bugs at compile time.

5.5 Lambda vs Anonymous Inner Class

Aspect	Anonymous Inner Class	Lambda Expression
Compilation	Generates separate .class file	Uses invokedynamic, no extra class file
`this` keyword	Refers to the anonymous class instance	Refers to the enclosing class instance
Interfaces	Can implement any interface (multiple methods)	Only functional interfaces (single abstract method)
State	Can have fields and mutable state	Stateless (captures effectively final variables)
Verbosity	Very verbose (type declarations, method signature)	Concise (type inference, minimal syntax)
Performance	Object creation overhead on each use	Can be cached and reused by the JVM
Shadowing	Can shadow enclosing scope variables	Cannot shadow enclosing scope variables

5.6 The `this` Keyword Difference

```
public class ThisExample {
    private String name = "Outer";

    public void demonstrateThis() {
        // Anonymous inner class: 'this' refers to the anonymous instance
        Runnable anon = new Runnable() {
            private String name = "Anonymous";
            @Override
            public void run() {
                System.out.println(this.name); // prints "Anonymous"
            }
        };

        // Lambda: 'this' refers to the enclosing class (ThisExample)
        Runnable lambda = () -> {
            System.out.println(this.name); // prints "Outer"
        };
    }
}
```

5.7 Real-World Enterprise Use Cases

Use Case 1: Spring Boot – Custom Validation

```
@Service
public class OrderValidationService {

    private final List<Predicate<Order>> validationRules = List.of(
        order -> order.getTotal() > 0,
        order -> order.getItems() != null && !order.getItems().isEmpty(),
        order -> order.getCustomerId() != null,
        order -> order.getShippingAddress() != null
    );

    public ValidationResult validate(Order order) {
        List<String> errors = validationRules.stream()
            .filter(rule -> !rule.test(order))
            .map(rule -> "Validation rule failed: " + rule.toString())
            .collect(Collectors.toList());

        return errors.isEmpty()
            ? ValidationResult.success()
            : ValidationResult.failure(errors);
    }
}
```

Use Case 2: Strategy Pattern with Lambdas

```
@Service
public class PricingService {

    private final Map<CustomerType, UnaryOperator<BigDecimal>> pricingStrategies;

    public PricingService() {
        pricingStrategies = Map.of(
            CustomerType.REGULAR,  price -> price,
            CustomerType.PREMIUM,  price -> price.multiply(BigDecimal.valueOf(0.90)),
            CustomerType.VIP,      price -> price.multiply(BigDecimal.valueOf(0.80)),
            CustomerType.EMPLOYEE, price -> price.multiply(BigDecimal.valueOf(0.70))
        );
    }

    public BigDecimal calculatePrice(BigDecimal basePrice, CustomerType type) {
        return pricingStrategies
            .getOrDefault(type, UnaryOperator.identity())
            .apply(basePrice);
    }
}
```

Use Case 3: Retry Logic with Lambda

```
public class RetryUtil {

    public static <T> T retry(Supplier<T> operation, int maxAttempts,
                              Duration delay, Predicate<Exception> retryOn) {
        Exception lastException = null;
        for (int attempt = 1; attempt ≤ maxAttempts; attempt++) {
            try {
                return operation.get();
            } catch (Exception e) {
                lastException = e;
                if (attempt == maxAttempts || !retryOn.test(e)) {
                    throw new RuntimeException("Failed after " + attempt + " attempts
                }
                try { Thread.sleep(delay.toMillis() * attempt); }
                catch (InterruptedException ie) { Thread.currentThread().interrupt();
            }
        }
        throw new RuntimeException("Unreachable", lastException);
    }
}

// Usage:
String result = RetryUtil.retry(
    () → externalApiClient.fetchData(id),          // Supplier
    3,                                              // max attempts
    Duration.ofSeconds(2),                          // delay
    e → e instanceof TimeoutException               // retry condition
);
```

5.8 Best Practices for Lambda Usage

✅ **Lambda Best Practices**

- **Keep lambdas short** – If a lambda exceeds 3-4 lines, extract it to a named method and use a method reference
- **Avoid side effects** – Lambdas in streams should be pure functions
- **Use method references** when possible – `String::length` is clearer than `s → s.length()`
- **Don't overuse lambdas** – Traditional loops may be clearer for simple operations with side effects
- **Avoid complex logic** in lambdas – Debugging is harder without named methods
- **Prefer standard functional interfaces** over creating custom ones

Chapter 6: Functional Interfaces

The type system foundation for lambda expressions

6.1 What is a Functional Interface?

A functional interface is an interface that has **exactly one abstract method**. It can have any number of default or static methods. Functional interfaces provide the target type for lambda expressions and method references.

```
@FunctionalInterface
public interface Calculator {
    double calculate(double a, double b);  // Single abstract method

    // Default methods are allowed
    default Calculator andThen(UnaryOperator<Double> after) {
        return (a, b) -> after.apply(this.calculate(a, b));
    }

    // Static methods are allowed
    static Calculator add() { return (a, b) -> a + b; }
    static Calculator multiply() { return (a, b) -> a * b; }
}
```

📘 **@FunctionalInterface Annotation**

This annotation is **optional** but strongly recommended. It tells the compiler to verify that the interface has exactly one abstract method. If someone accidentally adds a second abstract method, the compiler will generate an error. Think of it like `@Override` —not required, but prevents bugs.

6.2 Built-in Functional Interfaces (java.util.function)

Java 8 provides 43 built-in functional interfaces. The most important ones are:

6.2.1 Predicate<T>

What: Takes one argument, returns boolean. Used for testing/filtering conditions.

```
// Signature: boolean test(T t)
Predicate<String> isNotEmpty = s -> s != null && !s.isEmpty();
Predicate<Integer> isPositive = n -> n > 0;
Predicate<Employee> isActive = Employee::isActive;

// Usage in filtering
List<Employee> activeEmployees = employees.stream()
    .filter(isActive)
    .collect(Collectors.toList());

// Predicate chaining
Predicate<Employee> seniorActive = isActive
    .and(e -> e.getExperience() > 10)
    .and(e -> e.getSalary() > 80000);

Predicate<String> validEmail = s -> s.contains("@")
    .and(s2 -> s2.contains("."))
    .and(s3 -> s3.length() > 5);

// Negate
Predicate<Employee> inactiveEmployees = isActive.negate();

// BiPredicate
BiPredicate<String, Integer> hasMinLength = (str, len) -> str.length() >= len;
boolean result = hasMinLength.test("Hello", 3); // true
```

6.2.2 Function<T, R>

What: Takes one argument of type T, returns result of type R. Used for transformation/mapping.

```
// Signature: R apply(T t)
Function<String, Integer> toLength = String::length;
Function<Employee, String> toName = Employee::getName;
Function<String, String> toUpper = String::toUpperCase;

// Usage in mapping
List<String> names = employees.stream()
    .map(toName)
    .collect(Collectors.toList());

// Function composition with andThen (f.andThen(g) = g(f(x)))
Function<String, String> trimAndUpper = ((Function<String, String>) String::trim)
    .andThen(String::toUpperCase);
String result = trimAndUpper.apply("  hello  "); // "HELLO"

// Function composition with compose (f.compose(g) = f(g(x)))
Function<Integer, Integer> multiplyBy2 = x -> x * 2;
Function<Integer, Integer> add3 = x -> x + 3;

Function<Integer, Integer> add3ThenDouble = multiplyBy2.compose(add3);
int result = add3ThenDouble.apply(5); // (5 + 3) * 2 = 16

// Identity function
Function<String, String> identity = Function.identity();
// Useful as default/no-op transformation
```

6.2.3 Consumer<T>

What: Takes one argument, returns nothing. Used for side-effect operations (logging, saving, sending).

```
// Signature: void accept(T t)
Consumer<String> printer = System.out::println;
Consumer<Employee> saveToDb = employeeRepository::save;
Consumer<Order> sendNotification = order ->
    emailService.send(order.getCustomerEmail(), "Order confirmed: " + order.getId());

// Usage
employees.forEach(printer.andThen(saveToDb));

// Consumer chaining
Consumer<Order> processOrder = order -> orderService.validate(order);
processOrder = processOrder
    .andThen(order -> paymentService.charge(order))
    .andThen(order -> inventoryService.reserve(order))
    .andThen(order -> notificationService.notify(order));

orders.forEach(processOrder);

// BiConsumer
BiConsumer<String, Integer> printEntry = (key, value) ->
    System.out.println(key + " = " + value);
Map<String, Integer> scores = Map.of("Alice", 95, "Bob", 87);
scores.forEach(printEntry);
```

6.2.4 Supplier<T>

What: Takes no arguments, returns a value. Used for lazy evaluation, factories, and deferred computation.

```
// Signature: T get()
Supplier<LocalDateTime> now = LocalDateTime::now;
Supplier<UUID> idGenerator = UUID::randomUUID;
Supplier<List<String>> emptyList = ArrayList::new;
Supplier<Connection> dbConnection = () -> dataSource.getConnection();

// Lazy evaluation
public void processIfNeeded(boolean needed, Supplier<ExpensiveResult> computation) {
    if (needed) {
        ExpensiveResult result = computation.get(); // Only computed when needed
        process(result);
    }
}

// Factory pattern
Supplier<Employee> defaultEmployee = () -> {
    Employee e = new Employee();
    e.setStatus(Status.ACTIVE);
    e.setCreatedAt(LocalDateTime.now());
    e.setRole(Role.JUNIOR);
    return e;
};

Employee newHire = defaultEmployee.get();
```

6.2.5 UnaryOperator<T> and BinaryOperator<T>

What: Specialized forms of Function where input and output types are the same.

```
// UnaryOperator<T> extends Function<T, T>
UnaryOperator<String> toUpperCase = String::toUpperCase;
UnaryOperator<BigDecimal> applyTax = price →
    price.multiply(BigDecimal.valueOf(1.18)); // 18% GST
UnaryOperator<List<String>> sort = list → {
    Collections.sort(list);
    return list;
};

// Apply multiple transformations
List<UnaryOperator<String>> transformations = List.of(
    String::trim,
    String::toLowerCase,
    s → s.replaceAll("[^a-z0-9]", "-")
);
String result = transformations.stream()
    .reduce(UnaryOperator.identity(), (f, g) → f.andThen(g)::apply)
    .apply("  Hello World! ");  // "hello-world-"

// BinaryOperator<T> extends BiFunction<T, T, T>
BinaryOperator<Integer> add = Integer::sum;
BinaryOperator<Integer> max = Integer::max;
BinaryOperator<BigDecimal> multiply = BigDecimal::multiply;
BinaryOperator<String> concat = String::concat;

// Usage in reduce
int sum = numbers.stream().reduce(0, add);
Optional<Integer> maximum = numbers.stream().reduce(max);
```

6.2.6 BiFunction<T, U, R> and BiConsumer<T, U>

```
// BiFunction: Takes two arguments, returns a result
BiFunction<String, String, String> fullName = (first, last) → first + " " + last;
BiFunction<Integer, Integer, Double> divide = (a, b) → (double) a / b;
BiFunction<Employee, Double, Employee> giveRaise = (emp, percent) → {
    emp.setSalary(emp.getSalary() * (1 + percent / 100));
    return emp;
};

// Map.replaceAll uses BiFunction
Map<String, Integer> wordCount = new HashMap<>();
wordCount.replaceAll((word, count) → count + 1); // Increment all counts

// Map.computeIfAbsent uses Function
Map<String, List<Order>> ordersByCustomer = new HashMap<>();
ordersByCustomer.computeIfAbsent("customer-1", key → new ArrayList<>())
    .add(newOrder);
```

6.3 Functional Interface Summary Table

Interface	Method	Input	Output	Use Case
Predicate<T>	test(T)	T	boolean	Filtering, validation
Function<T,R>	apply(T)	T	R	Transformation, mapping
Consumer<T>	accept(T)	T	void	Side effects (print, save)
Supplier<T>	get()	none	T	Factory, lazy computation
UnaryOperator<T>	apply(T)	T	T	Same-type transformation
BinaryOperator<T>	apply(T,T)	T, T	T	Reduction, combining
BiFunction<T,U,R>	apply(T,U)	T, U	R	Two-arg transformation
BiPredicate<T,U>	test(T,U)	T, U	boolean	Two-arg condition
BiConsumer<T,U>	accept(T,U)	T, U	void	Map.forEach, two-arg actions

6.4 Primitive Specializations

To avoid autoboxing overhead, Java 8 provides primitive-specialized versions:

```
// IntPredicate, LongPredicate, DoublePredicate
IntPredicate isEven = n -> n % 2 == 0;

// IntFunction, LongFunction, DoubleFunction
IntFunction<String> intToString = Integer::toString;

// IntConsumer, LongConsumer, DoubleConsumer
IntConsumer printInt = System.out::println;

// IntSupplier, LongSupplier, DoubleSupplier
IntSupplier randomInt = () -> ThreadLocalRandom.current().nextInt(100);

// IntUnaryOperator, LongUnaryOperator, DoubleUnaryOperator
IntUnaryOperator doubleIt = n -> n * 2;

// IntBinaryOperator, LongBinaryOperator, DoubleBinaryOperator
IntBinaryOperator add = Integer::sum;

// ToIntFunction, ToLongFunction, ToDoubleFunction
ToIntFunction<String> length = String::length;

// ObjIntConsumer, ObjLongConsumer, ObjDoubleConsumer
ObjIntConsumer<List<Integer>> addAtIndex = List::add;
```

⚡ Performance Tip

Always use primitive specializations (`IntStream` , `IntPredicate` , etc.) when working with primitive types in streams. Autoboxing/unboxing can cause significant performance degradation in large-scale data processing. For example, `IntStream.range(0, 1000000).sum()` is much faster than `Stream.iterate(0, i -> i + 1).limit(1000000).reduce(0, Integer::sum)` .

6.5 Creating Custom Functional Interfaces

```
// Custom functional interface for business logic
@FunctionalInterface
public interface OrderProcessor {
    ProcessingResult process(Order order) throws ProcessingException;

    // Chain processors
    default OrderProcessor andThen(OrderProcessor next) {
        return order -> {
            ProcessingResult result = this.process(order);
            if (result.isSuccess()) {
                return next.process(order);
            }
            return result;
        };
    }

    // Static factory methods
    static OrderProcessor validate() {
        return order -> {
            if (order.getItems().isEmpty()) {
                return ProcessingResult.failure("No items in order");
            }
            return ProcessingResult.success();
        };
    }
}

// Usage
OrderProcessor pipeline = OrderProcessor.validate()
    .andThen(order -> paymentService.charge(order))
    .andThen(order -> inventoryService.reserve(order))
    .andThen(order -> shippingService.schedule(order));

ProcessingResult result = pipeline.process(newOrder);
```

6.6 Functional Interfaces in Spring Boot

```
// Spring's functional bean registration (Spring 5+)
@SpringBootApplication
public class Application {
    public static void main(String[] args) {
        SpringApplication app = new SpringApplication(Application.class);
        app.addInitializers((GenericApplicationContext ctx) -> {
            ctx.registerBean(OrderService.class,
                () -> new OrderService(ctx.getBean(OrderRepository.class)));
            ctx.registerBean(NotificationService.class,
                () -> new NotificationService(ctx.getBean(EmailClient.class)));
        });
        app.run(args);
    }
}

// Spring WebFlux functional endpoints
@Configuration
public class RouterConfig {
    @Bean
    RouterFunction<ServerResponse> routes(OrderHandler handler) {
        return RouterFunctions.route()
            .GET("/orders", handler::getAll)
            .GET("/orders/{id}", handler::getById)
            .POST("/orders", handler::create)
            .PUT("/orders/{id}", handler::update)
            .DELETE("/orders/{id}", handler::delete)
            .build();
    }
}
```

Chapter 7: Method References

Shorthand notation for calling existing methods via lambdas

7.1 What are Method References?

Method references are a shorthand notation for lambdas that simply call an existing method. They make code more readable by using method names directly instead of lambda expressions. A method reference is the lambda equivalent of calling a method by name.

7.2 Types of Method References

Type	Syntax	Lambda Equivalent	Example
Static Method	ClassName::staticMethod	(args) → ClassName.staticMethod(args)	Integer::parseInt
Instance Method (specific object)	object::instanceMethod	(args) → object.instanceMethod(args)	System.out::println
Instance Method (arbitrary object)	ClassName::instanceMethod	(obj, args) → obj.instanceMethod(args)	String::toUpperCase
Constructor	ClassName::new	(args) → new ClassName(args)	ArrayList::new

7.3 Static Method References

```
// Lambda → Method Reference
Function<String, Integer> parseInt = s -> Integer.parseInt(s);
Function<String, Integer> parseInt = Integer::parseInt;  // ✅ Cleaner

// More examples
Function<Double, Long> round = Math::round;
Function<String, Boolean> isNull = Objects::isNull;
BinaryOperator<Integer> max = Math::max;
Function<String, String> valueOf = String::valueOf;

// In streams
List<Integer> numbers = strings.stream()
    .map(Integer::parseInt)        // Static method reference
    .collect(Collectors.toList());

// Custom static method
public class Validator {
    public static boolean isValidEmail(String email) {
        return email != null && email.matches("^[\\w.]+@[\\w.]+\\.[a-z]{2,}$");
    }
}
long validCount = emails.stream()
    .filter(Validator::isValidEmail)   // Static method reference
    .count();
```

7.4 Instance Method References (Specific Object)

```
// Reference to a method of a specific object instance
PrintStream out = System.out;
Consumer<String> printer = out::println;

// Common pattern: service method reference
OrderService orderService = new OrderService();
Consumer<Order> processor = orderService::processOrder;
orders.forEach(processor);

// String builder
StringBuilder sb = new StringBuilder();
Consumer<String> appender = sb::append;
words.forEach(appender);
String result = sb.toString();

// Repository save
EmployeeRepository repo = getRepository();
Consumer<Employee> save = repo::save;
employees.forEach(save);

// Logger
Logger logger = LoggerFactory.getLogger(MyClass.class);
Consumer<String> logInfo = logger::info;
Consumer<String> logError = logger::error;
```

7.5 Instance Method References (Arbitrary Object of a Type)

```
// The first parameter becomes the object on which the method is invoked
Function<String, String> toUpper = String::toUpperCase;
// Equivalent to: s -> s.toUpperCase()

Function<String, Integer> length = String::length;
// Equivalent to: s -> s.length()

Predicate<String> isEmpty = String::isEmpty;
// Equivalent to: s -> s.isEmpty()

// Comparator example
Comparator<String> natural = String::compareTo;
// Equivalent to: (s1, s2) -> s1.compareTo(s2)

// Two parameter example
BiPredicate<String, String> startsWith = String::startsWith;
// Equivalent to: (s, prefix) -> s.startsWith(prefix)

// Chained stream operations
List<String> upperNames = employees.stream()
    .map(Employee::getName)           // Arbitrary object method ref
    .map(String::toUpperCase)         // Arbitrary object method ref
    .filter(Predicate.not(String::isEmpty))  // Predicate.not (Java 11+)
    .sorted(String::compareTo)        // Arbitrary object method ref
    .collect(Collectors.toList());
```

7.6 Constructor References

```
// No-arg constructor
Supplier<List<String>> listFactory = ArrayList::new;
// Equivalent to: () → new ArrayList<>()

Supplier<StringBuilder> sbFactory = StringBuilder::new;

// Single-arg constructor
Function<String, Employee> employeeFactory = Employee::new;
// Requires Employee(String name) constructor

// Two-arg constructor
BiFunction<String, Integer, Employee> empFactory = Employee::new;
// Requires Employee(String name, int age) constructor

// Array constructor
Function<Integer, String[]> arrayFactory = String[]::new;
String[] array = arrayFactory.apply(10); // Creates String[10]

// Commonly used with toArray()
String[] names = namesList.stream()
    .toArray(String[]::new);

// Factory pattern in enterprise code
@Configuration
public class AppConfig {
    @Bean
    public Function<OrderDTO, Order> orderFactory() {
        return Order::new;  // Constructor reference as bean
    }

    @Bean
    public Supplier<AuditLog> auditLogFactory() {
        return AuditLog::new;
    }
}
```

7.7 When to Use Method References vs Lambdas

Scenario	Prefer	Example
Simple method call	Method Reference	`String::length` over `s -> s.length()`
Method with additional logic	Lambda	`s -> s.length() > 5`
Multiple parameters used	Lambda	`(a, b) -> a.concat(b).trim()`
Readability matters	Method Reference	`Employee::getName` is self-documenting
Complex transformations	Named method + Reference	Extract to `this::transformEmployee`

💡 **Pro Tip: Extract Complex Lambdas**

When a lambda expression grows beyond 2-3 lines, extract it into a private method and use a method reference. This improves readability, testability, and allows you to give meaningful names to operations.

```
// Instead of complex inline lambda:
.map(order -> {
    OrderDTO dto = new OrderDTO();
    dto.setId(order.getId());
    dto.setTotal(order.getTotal());
    dto.setStatus(order.getStatus().name());
    dto.setCustomerName(order.getCustomer().getName());
    return dto;
})

// Extract to method and use method reference:
.map(this::toOrderDTO)

private OrderDTO toOrderDTO(Order order) {
    OrderDTO dto = new OrderDTO();
    dto.setId(order.getId());
    dto.setTotal(order.getTotal());
    dto.setStatus(order.getStatus().name());
    dto.setCustomerName(order.getCustomer().getName());
    return dto;
}
```

7.8 Real-World Enterprise Patterns with Method References

```
// Pattern 1: Repository layer
@Service
public class UserService {
    @Autowired
    private UserRepository userRepository;

    public Optional<UserDTO> findById(Long id) {
        return userRepository.findById(id)       // Returns Optional<User>
            .map(UserMapper::toDTO);              // Constructor/static method ref
    }

    public List<UserDTO> findActiveUsers() {
        return userRepository.findAll().stream()
            .filter(User::isActive)               // Instance method ref
            .map(UserMapper::toDTO)               // Static method ref
            .sorted(Comparator.comparing(UserDTO::getName))  // Instance method ref
            .collect(Collectors.toList());
    }
}

// Pattern 2: Event handling
@Component
public class OrderEventPublisher {
    private final List<Consumer<OrderEvent>> listeners = new ArrayList<>();

    public void subscribe(Consumer<OrderEvent> listener) {
        listeners.add(listener);
    }

    public void publish(OrderEvent event) {
        listeners.forEach(listener -> listener.accept(event));
    }
}

// Registration with method references
eventPublisher.subscribe(auditService::logEvent);
eventPublisher.subscribe(notificationService::sendNotification);
eventPublisher.subscribe(analyticsService::trackEvent);
```

Chapter 8: Streams API – Fundamentals

The most powerful data processing tool in modern Java

8.1 What is a Stream?

A Stream is a **sequence of elements** that supports sequential and parallel aggregate operations. Unlike collections, streams do not store data—they convey elements from a source (collection, array, generator function, or I/O channel) through a pipeline of computational operations.

📘 **Key Characteristics of Streams**

- **Not a data structure** – Streams don't store elements; they carry them from source to result
- **Functional in nature** – Operations produce results without modifying the source
- **Lazily evaluated** – Intermediate operations are not executed until a terminal operation is invoked
- **Possibly unbounded** – Streams can be infinite (e.g., `Stream.generate()`)
- **Consumable** – Elements are visited only once during the life of a stream; a new stream must be generated to revisit

8.2 Stream vs Collection

Aspect	Collection	Stream
Data Storage	Stores all elements in memory	Computes elements on demand
Iteration	External (you manage the iterator)	Internal (stream manages iteration)
Traversal	Can be traversed multiple times	Can be traversed only once
Eagerness	Eagerly constructed	Lazily constructed
Modification	Can add/remove elements	Cannot modify source
Size	Finite	Can be infinite
Parallelism	Manual thread management	Built-in parallel support

8.3 Stream Creation

```
// 1. From Collection
List<String> names = Arrays.asList("Alice", "Bob", "Charlie");
Stream<String> stream1 = names.stream();
Stream<String> parallelStream = names.parallelStream();

// 2. From Array
String[] array = {"A", "B", "C"};
Stream<String> stream2 = Arrays.stream(array);
Stream<String> stream3 = Arrays.stream(array, 0, 2); // "A", "B"

// 3. Stream.of()
Stream<String> stream4 = Stream.of("X", "Y", "Z");
Stream<Integer> stream5 = Stream.of(1, 2, 3, 4, 5);

// 4. Stream.empty()
Stream<String> empty = Stream.empty();

// 5. Stream.generate() - Infinite stream
Stream<UUID> uuids = Stream.generate(UUID::randomUUID);
Stream<Double> randoms = Stream.generate(Math::random);

// 6. Stream.iterate() - Infinite stream with seed
Stream<Integer> counting = Stream.iterate(0, n -> n + 1);
Stream<Integer> powersOf2 = Stream.iterate(1, n -> n * 2);

// 7. From String
IntStream chars = "Hello World".chars();

// 8. From Files
Stream<String> lines = Files.lines(Paths.get("data.txt"));

// 9. Primitive Streams
IntStream intStream = IntStream.range(1, 100);       // 1 to 99
IntStream intStream2 = IntStream.rangeClosed(1, 100); // 1 to 100
LongStream longStream = LongStream.rangeClosed(1, 1000000);
DoubleStream doubleStream = DoubleStream.of(1.1, 2.2, 3.3);

// 10. Stream.Builder
Stream<String> built = Stream.<String>builder()
    .add("one")
    .add("two")
    .add("three")
    .build();

// 11. From Map
Map<String, Integer> map = Map.of("a", 1, "b", 2);
Stream<Map.Entry<String, Integer>> entryStream = map.entrySet().stream();
```

8.4 Stream Pipeline Architecture

```
Stream Pipeline = Source → Intermediate Operations → Terminal Operation
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ │ Source │──▶│ filter
│──▶│ map │──▶│ sorted │──▶│ collect │ │(Collection)│ │(Predicate)│ │(Function)│
│(Comparator)│ │(Terminal)│ └──────────┘ └──────────┘ └──────────┘ └──────────┘
└──────────┘ LAZY LAZY LAZY EAGER (Intermediate) (Intermediate) (Intermediate)
(Terminal)
```

8.5 Lazy Evaluation – How It Works

Intermediate operations are lazy—they set up the pipeline but don't process any elements until a terminal operation triggers execution. This enables powerful optimizations:

```java
// Nothing happens until collect() is called
Stream<String> pipeline = employees.stream()
    .filter(e -> {
        System.out.println("Filtering: " + e.getName());
        return e.getSalary() > 50000;
    })
    .map(e -> {
        System.out.println("Mapping: " + e.getName());
        return e.getName();
    });
// No output yet! The stream is lazy.

// Terminal operation triggers execution
List<String> result = pipeline.collect(Collectors.toList());
// NOW the filter and map operations execute
```

⚡ Short-Circuit Optimization

Some terminal operations are short-circuiting—they can produce a result without processing all elements. Combined with lazy evaluation, this provides excellent performance:

```
// Only processes elements until first match is found
Optional<Employee> first = employees.stream()
    .filter(e -> e.getSalary() > 100000)  // Stops filtering once found
    .findFirst();                          // Short-circuit terminal op

// With limit(), processes only enough elements to fill the limit
List<String> topThreeNames = employees.stream()
    .sorted(Comparator.comparing(Employee::getSalary).reversed())
    .limit(3)                              // Only need 3 elements
    .map(Employee::getName)
    .collect(Collectors.toList());

// Loop fusion: filter + map are combined into single pass
// The stream does NOT first filter ALL elements, then map ALL results
// It processes each element through the entire pipeline before moving to the next
```

8.6 Internal vs External Iteration

```
// EXTERNAL ITERATION (Pre-Java 8)
// Developer controls the iteration mechanism
Iterator<Employee> iterator = employees.iterator();
List<String> names = new ArrayList<>();
while (iterator.hasNext()) {
    Employee emp = iterator.next();
    if (emp.isActive()) {
        names.add(emp.getName());
    }
}

// INTERNAL ITERATION (Java 8 Streams)
// Stream library handles iteration
List<String> names = employees.stream()
    .filter(Employee::isActive)
    .map(Employee::getName)
    .collect(Collectors.toList());

// Benefits of Internal Iteration:
// 1. Parallelism is transparent - just call .parallelStream()
// 2. Loop fusion - multiple operations optimized together
// 3. Short-circuiting - stops early when possible
// 4. Lazy evaluation - only processes what's needed
```

8.7 Stream Operation Categories

Category	Operations	Returns	Evaluation
Intermediate (Stateless)	filter, map, flatMap, peek	Stream	Lazy
Intermediate (Stateful)	distinct, sorted, limit, skip	Stream	Lazy (but may buffer)
Terminal (Non-short-circuit)	forEach, collect, reduce, count, toArray	Result	Eager
Terminal (Short-circuit)	findFirst, findAny, anyMatch, allMatch, noneMatch	Result	Eager (may stop early)

⚠️ **Common Mistake: Reusing Streams**

A stream can only be consumed once. Attempting to reuse a stream throws `IllegalStateException`.

```
Stream<String> stream = names.stream();
stream.forEach(System.out::println);  // OK - first use
stream.forEach(System.out::println);  // ❌ IllegalStateException!

// Solution: Create a Supplier of streams
Supplier<Stream<String>> streamSupplier = () -> names.stream();
streamSupplier.get().forEach(System.out::println);  // OK
streamSupplier.get().filter(n -> n.length() > 3).count();  // OK
```

Chapter 9: Stream Operations Deep Dive

Mastering every intermediate and terminal operation

9.1 Intermediate Operations

9.1.1 filter(Predicate<T>)

Returns a stream consisting of elements that match the given predicate.

```
// Basic filtering
List<Employee> seniors = employees.stream()
    .filter(e -> e.getExperience() > 10)
    .collect(Collectors.toList());

// Complex predicate
List<Order> urgentOrders = orders.stream()
    .filter(o -> o.getStatus() == Status.PENDING)
    .filter(o -> o.getPriority() == Priority.HIGH)
    .filter(o -> o.getDeadline().isBefore(LocalDate.now().plusDays(3)))
    .collect(Collectors.toList());

// Using composed predicates
Predicate<Product> inStock = p -> p.getQuantity() > 0;
Predicate<Product> onSale = Product::isOnSale;
Predicate<Product> affordable = p -> p.getPrice() < 100;

List<Product> deals = products.stream()
    .filter(inStock.and(onSale).and(affordable))
    .collect(Collectors.toList());
```

9.1.2 map(Function<T, R>)

Transforms each element by applying a function, producing a stream of the results.

```
// Simple transformation
List<String> names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toList());

// Type conversion
List<EmployeeDTO> dtos = employees.stream()
    .map(emp -> new EmployeeDTO(emp.getId(), emp.getName(), emp.getEmail()))
    .collect(Collectors.toList());

// Chain of maps
List<String> initials = employees.stream()
    .map(Employee::getName)
    .map(name -> name.substring(0, 1))
    .map(String::toUpperCase)
    .collect(Collectors.toList());

// Primitive mapping (avoids autoboxing)
int totalLength = names.stream()
    .mapToInt(String::length)    // Returns IntStream
    .sum();                       // Efficient primitive sum

double avgSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0.0);
```

9.1.3 flatMap(Function<T, Stream<R>>)

Transforms each element into a stream, then flattens all resulting streams into a single stream. Essential for handling nested structures.

```java
// Flatten nested lists
List<List<String>> nestedNames = List.of(
    List.of("Alice", "Bob"),
    List.of("Charlie", "David"),
    List.of("Eve")
);
List<String> allNames = nestedNames.stream()
    .flatMap(Collection::stream)       // Flatten
    .collect(Collectors.toList());
// Result: ["Alice", "Bob", "Charlie", "David", "Eve"]

// Orders → all order items
List<OrderItem> allItems = orders.stream()
    .flatMap(order -> order.getItems().stream())
    .collect(Collectors.toList());

// Departments → all employees across departments
List<Employee> allEmployees = departments.stream()
    .flatMap(dept -> dept.getEmployees().stream())
    .filter(Employee::isActive)
    .collect(Collectors.toList());

// Split strings into words
List<String> sentences = List.of("Hello World", "Java 8 Streams");
List<String> words = sentences.stream()
    .flatMap(s -> Arrays.stream(s.split(" ")))
    .collect(Collectors.toList());
// Result: ["Hello", "World", "Java", "8", "Streams"]

// Optional flatMap
Optional<String> city = Optional.ofNullable(user)
    .flatMap(User::getAddress)     // Returns Optional<Address>
    .flatMap(Address::getCity)     // Returns Optional<City>
    .map(City::getName);
```

📘 map vs flatMap

`map` applies a function to each element and wraps the result (1-to-1 mapping). `flatMap` applies a function that returns a stream for each element, then merges all streams into one (1-to-many mapping). Use `flatMap` when each element can produce zero, one, or many results.

9.1.4 distinct()

Returns a stream with duplicate elements removed (uses `equals()`).

```
List<String> unique = names.stream()
    .distinct()
    .collect(Collectors.toList());

// Distinct by property (requires custom logic)
List<Employee> uniqueByDept = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        (e1, e2) -> e1       // Keep first occurrence
    ))
    .values().stream()
    .collect(Collectors.toList());
```

9.1.5 sorted()

```
// Natural order
List<String> sorted = names.stream()
    .sorted()
    .collect(Collectors.toList());

// Custom comparator
List<Employee> bySalaryDesc = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .collect(Collectors.toList());

// Multiple sort criteria
List<Employee> sorted = employees.stream()
    .sorted(Comparator
        .comparing(Employee::getDepartment)
        .thenComparing(Employee::getSalary, Comparator.reverseOrder())
        .thenComparing(Employee::getName))
    .collect(Collectors.toList());

// Null-safe sorting
List<Employee> sorted = employees.stream()
    .sorted(Comparator.comparing(
        Employee::getManager,
        Comparator.nullsLast(Comparator.naturalOrder())))
    .collect(Collectors.toList());
```

9.1.6 limit(long) and skip(long)

```
// Pagination using skip and limit
int page = 2;
int pageSize = 10;
List<Employee> pageResult = employees.stream()
    .sorted(Comparator.comparing(Employee::getName))
    .skip((long) (page - 1) * pageSize)    // Skip first page
    .limit(pageSize)                        // Take one page
    .collect(Collectors.toList());

// Top 5 highest salaries
List<Employee> top5 = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .limit(5)
    .collect(Collectors.toList());

// Skip header rows in data processing
List<String[]> data = Files.lines(Paths.get("data.csv"))
    .skip(1)                                // Skip header
    .map(line -> line.split(","))
    .collect(Collectors.toList());
```

9.1.7 peek(Consumer<T>)

Performs an action on each element as it flows through. Primarily for debugging.

```
// Debugging stream pipeline
List<String> result = employees.stream()
    .filter(e -> e.getSalary() > 50000)
    .peek(e -> System.out.println("After filter: " + e.getName()))
    .map(Employee::getName)
    .peek(name -> System.out.println("After map: " + name))
    .sorted()
    .peek(name -> System.out.println("After sort: " + name))
    .collect(Collectors.toList());
```

> ⚠️ **peek() Warning**
>
> Do NOT use `peek()` for side effects in production code. Its behavior may change with parallel streams or if the JVM optimizes away certain intermediate operations. Use it only for debugging. For side effects, use `forEach()` terminal operation or extract the logic.

9.2 Terminal Operations

9.2.1 collect(Collector)

The most versatile terminal operation. Reduces stream elements into a result container (List, Set, Map, String, etc.). Covered in detail in Chapter 10.

```
List<String> list = stream.collect(Collectors.toList());
Set<String> set = stream.collect(Collectors.toSet());
Map<Long, Employee> map = stream.collect(Collectors.toMap(Employee::getId, e -> e));
```

9.2.2 reduce()

Combines all elements into a single result using an associative accumulation function.

```
// Sum with identity
int sum = numbers.stream().reduce(0, Integer::sum);

// Sum without identity (returns Optional)
Optional<Integer> sum = numbers.stream().reduce(Integer::sum);

// Max salary
Optional<Double> maxSalary = employees.stream()
    .map(Employee::getSalary)
    .reduce(Double::max);

// String concatenation
String allNames = names.stream()
    .reduce("", (a, b) -> a.isEmpty() ? b : a + ", " + b);

// Complex reduce: calculating total order value
BigDecimal totalValue = orders.stream()
    .map(Order::getTotal)
    .reduce(BigDecimal.ZERO, BigDecimal::add);

// Three-argument reduce (for parallel streams)
int totalLength = strings.stream()
    .reduce(
        0,                          // identity
        (total, str) -> total + str.length(),  // accumulator
        Integer::sum                // combiner (for parallel)
    );
```

📘 **reduce vs collect**

`reduce` is for combining elements into a single immutable value (sum, max, concatenation). `collect` is for accumulating elements into a mutable container (List, Map, StringBuilder). Use `collect` when building collections; use `reduce` for aggregation.

9.2.3 forEach() and forEachOrdered()

```
// forEach - no guaranteed order in parallel streams
employees.stream().forEach(System.out::println);

// forEachOrdered - maintains encounter order even in parallel
employees.parallelStream()
    .forEachOrdered(System.out::println);

// Side effect operations
orders.stream()
    .filter(o -> o.getStatus() == Status.PENDING)
    .forEach(orderService::process);
```

9.2.4 count(), min(), max()

```
long count = employees.stream()
    .filter(Employee::isActive)
    .count();

Optional<Employee> youngest = employees.stream()
    .min(Comparator.comparing(Employee::getAge));

Optional<Employee> highestPaid = employees.stream()
    .max(Comparator.comparingDouble(Employee::getSalary));

// Primitive streams have direct methods
int maxSalary = employees.stream()
    .mapToInt(e -> (int) e.getSalary())
    .max()
    .orElse(0);

IntSummaryStatistics stats = employees.stream()
    .mapToInt(e -> (int) e.getSalary())
    .summaryStatistics();
// stats.getMax(), stats.getMin(), stats.getAverage(), stats.getSum(), stats.getCount
```

9.2.5 findFirst() and findAny()

```
// findFirst - returns first element (respects encounter order)
Optional<Employee> first = employees.stream()
    .filter(e -> e.getDepartment().equals("Engineering"))
    .findFirst();

// findAny - returns any element (faster in parallel streams)
Optional<Employee> any = employees.parallelStream()
    .filter(e -> e.getSalary() > 100000)
    .findAny();

// Pattern: findFirst with orElseThrow
Employee manager = employees.stream()
    .filter(e -> e.getRole() == Role.MANAGER)
    .findFirst()
    .orElseThrow(() -> new NotFoundException("No manager found"));
```

9.2.6 anyMatch(), allMatch(), noneMatch()

```
// anyMatch - true if any element matches
boolean hasHighEarner = employees.stream()
    .anyMatch(e -> e.getSalary() > 200000);

// allMatch - true if ALL elements match
boolean allActive = employees.stream()
    .allMatch(Employee::isActive);

// noneMatch - true if NO elements match
boolean noInterns = employees.stream()
    .noneMatch(e -> e.getRole() == Role.INTERN);

// Enterprise validation example
boolean isValidOrder = order.getItems().stream()
    .allMatch(item -> item.getQuantity() > 0
        && item.getPrice().compareTo(BigDecimal.ZERO) > 0);
```

9.2.7 toArray()

```
// To Object array
Object[] objArray = names.stream().toArray();

// To typed array using constructor reference
String[] strArray = names.stream().toArray(String[]::new);

// To primitive array
int[] salaries = employees.stream()
    .mapToInt(e -> (int) e.getSalary())
    .toArray();
```

Chapter 10: Collectors

Powerful reduction operations for aggregating stream results

10.1 What are Collectors?

Collectors are reduction operations that accumulate stream elements into a summary result—a collection, a string, a statistical summary, or a grouped structure. The `Collectors` utility class provides a rich set of pre-built collectors for common operations.

10.2 Collection Collectors

toList(), toSet(), toCollection()

```
// toList - returns ArrayList
List<String> names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toList());

// toUnmodifiableList (Java 10+)
List<String> immutableNames = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toUnmodifiableList());

// toSet - returns HashSet (removes duplicates)
Set<String> departments = employees.stream()
    .map(Employee::getDepartment)
    .collect(Collectors.toSet());

// toCollection - specify exact collection type
TreeSet<String> sortedNames = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.toCollection(TreeSet::new));

LinkedList<Employee> linkedList = employees.stream()
    .collect(Collectors.toCollection(LinkedList::new));
```

toMap()

```
// Basic toMap
Map<Long, Employee> employeeById = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));

// toMap with value extraction
Map<Long, String> nameById = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Employee::getName));

// Handling duplicate keys with merge function
Map<String, Employee> highestPaidByDept = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,                    // key
        Function.identity(),                         // value
        (e1, e2) -> e1.getSalary() > e2.getSalary() ? e1 : e2  // merge
    ));

// Specifying map implementation
LinkedHashMap<Long, String> orderedMap = employees.stream()
    .sorted(Comparator.comparing(Employee::getName))
    .collect(Collectors.toMap(
        Employee::getId,
        Employee::getName,
        (v1, v2) -> v1,       // merge function (shouldn't occur with unique IDs)
        LinkedHashMap::new     // map factory
    ));

// ConcurrentMap for thread safety
ConcurrentMap<Long, Employee> concurrentMap = employees.parallelStream()
    .collect(Collectors.toConcurrentMap(Employee::getId, Function.identity()));
```

10.3 Grouping Collectors

groupingBy()

```
// Simple grouping
Map<String, List<Employee>> byDepartment = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Group and count
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment, Collectors.counting()));

// Group and sum
Map<String, Double> totalSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.summingDouble(Employee::getSalary)
    ));

// Group and get average
Map<String, Double> avgSalaryByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.averagingDouble(Employee::getSalary)
    ));

// Group and collect names
Map<String, List<String>> namesByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));

// Group and join names
Map<String, String> nameStringByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(
            Employee::getName,
            Collectors.joining(", ")
        )
    ));

// Multi-level grouping
Map<String, Map<String, List<Employee>>> byDeptAndRole = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.groupingBy(e -> e.getRole().name())
    ));

// Group with sorted map
Map<String, List<Employee>> sortedByDept = employees.stream()
```

```
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        TreeMap::new,          // Use TreeMap for sorted keys
        Collectors.toList()
    ));

// Group and get max
Map<String, Optional<Employee>> highestPaidByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));
```

partitioningBy()

Special case of grouping that divides elements into exactly two groups based on a predicate (true/false).

```
// Partition into active/inactive
Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(Employee::isActive));
List<Employee> activeEmps = partitioned.get(true);
List<Employee> inactiveEmps = partitioned.get(false);

// Partition and count
Map<Boolean, Long> countByStatus = employees.stream()
    .collect(Collectors.partitioningBy(
        e -> e.getSalary() > 75000,
        Collectors.counting()
    ));

// Partition with downstream collector
Map<Boolean, List<String>> namesByEligibility = employees.stream()
    .collect(Collectors.partitioningBy(
        e -> e.getExperience() >= 5,
        Collectors.mapping(Employee::getName, Collectors.toList())
    ));
```

10.4 Statistical Collectors

```
// counting
long count = employees.stream()
    .collect(Collectors.counting());

// summarizingInt/Long/Double
DoubleSummaryStatistics stats = employees.stream()
    .collect(Collectors.summarizingDouble(Employee::getSalary));
System.out.println("Count: " + stats.getCount());
System.out.println("Sum: " + stats.getSum());
System.out.println("Min: " + stats.getMin());
System.out.println("Max: " + stats.getMax());
System.out.println("Avg: " + stats.getAverage());

// averagingInt/Long/Double
double avgSalary = employees.stream()
    .collect(Collectors.averagingDouble(Employee::getSalary));

// summingInt/Long/Double
double totalSalary = employees.stream()
    .collect(Collectors.summingDouble(Employee::getSalary));

// minBy / maxBy
Optional<Employee> youngest = employees.stream()
    .collect(Collectors.minBy(Comparator.comparing(Employee::getAge)));

Optional<Employee> highest = employees.stream()
    .collect(Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary)));
```

10.5 String Collectors

```
// joining
String names = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.joining());          // "AliceBobCharlie"

String csv = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.joining(", "));      // "Alice, Bob, Charlie"

String formatted = employees.stream()
    .map(Employee::getName)
    .collect(Collectors.joining(", ", "[", "]")); // "[Alice, Bob, Charlie]"

// SQL IN clause generation
String inClause = ids.stream()
    .map(String::valueOf)
    .collect(Collectors.joining(", ", "WHERE id IN (", ")"));
// "WHERE id IN (1, 2, 3, 4, 5)"
```

10.6 Advanced Collectors

mapping() and flatMapping()

```
// mapping - transform before collecting
Map<String, Set<String>> skillsByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.mapping(Employee::getPrimarySkill, Collectors.toSet())
    ));

// flatMapping (Java 9+) - flatten and collect
Map<String, Set<String>> allSkillsByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.flatMapping(
            e -> e.getSkills().stream(),
            Collectors.toSet()
        )
    ));
```

collectingAndThen()

```
// Apply a final transformation after collecting
List<Employee> unmodifiable = employees.stream()
    .filter(Employee::isActive)
    .collect(Collectors.collectingAndThen(
        Collectors.toList(),
        Collections::unmodifiableList
    ));

// Calculate average then round
double roundedAvg = employees.stream()
    .collect(Collectors.collectingAndThen(
        Collectors.averagingDouble(Employee::getSalary),
        avg -> Math.round(avg * 100.0) / 100.0
    ));
```

reducing()

```
// Sum of salaries using reducing
Optional<Double> totalSalary = employees.stream()
    .map(Employee::getSalary)
    .collect(Collectors.reducing(Double::sum));

// With identity
double total = employees.stream()
    .collect(Collectors.reducing(0.0, Employee::getSalary, Double::sum));
```

📏 Collector Selection Guide

Need	Collector
Elements → List	toList()
Elements → Set (unique)	toSet()
Elements → Map	toMap()
Group by property	groupingBy()
Split into true/false	partitioningBy()
Count elements	counting()
Join strings	joining()
Get statistics	summarizingInt/Double/Long()
Post-process result	collectingAndThen()

Chapter 11: Optional Class

Eliminating NullPointerException with type-safe null handling

11.1 What is Optional?

Optional is a container object that may or may not contain a non-null value. It was introduced to provide a better alternative to returning `null` from methods, forcing callers to explicitly handle the absence of a value. It's Java's answer to the billion-dollar mistake—null references.

11.2 Creating Optional

```
// Empty Optional
Optional<String> empty = Optional.empty();

// Of (throws NullPointerException if null)
Optional<String> name = Optional.of("Alice");
Optional<String> nullVal = Optional.of(null); // ❌ NullPointerException!

// OfNullable (allows null → becomes empty Optional)
Optional<String> maybeName = Optional.ofNullable(getName()); // Safe
Optional<String> maybeNull = Optional.ofNullable(null);      // Optional.empty()
```

11.3 Checking and Retrieving Values

```
Optional<Employee> empOpt = repository.findById(id);

// isPresent() / isEmpty() (isEmpty is Java 11+)
if (empOpt.isPresent()) {
    Employee emp = empOpt.get();
    System.out.println(emp.getName());
}

// get() - throws NoSuchElementException if empty
Employee emp = empOpt.get(); // ❌ Dangerous without isPresent check

// orElse - provide default value (always evaluated)
Employee emp = empOpt.orElse(new Employee("Default"));

// orElseGet - lazy default (evaluated only when empty)
Employee emp = empOpt.orElseGet(() -> employeeService.getDefault());

// orElseThrow - throw exception if empty
Employee emp = empOpt.orElseThrow(
    () -> new EmployeeNotFoundException("Employee not found: " + id));

// ifPresent - execute action only when present
empOpt.ifPresent(e -> emailService.sendWelcome(e.getEmail()));

// ifPresentOrElse (Java 9+)
empOpt.ifPresentOrElse(
    e -> System.out.println("Found: " + e.getName()),
    () -> System.out.println("Employee not found")
);
```

11.4 Transforming Optional Values

```
// map - transform value if present
Optional<String> name = empOpt.map(Employee::getName);
Optional<String> upperName = empOpt
    .map(Employee::getName)
    .map(String::toUpperCase);

// flatMap - when transformation returns Optional
// Prevents Optional<Optional<T>> nesting
Optional<String> city = empOpt
    .flatMap(Employee::getAddress)          // Returns Optional<Address>
    .flatMap(Address::getCity)              // Returns Optional<City>
    .map(City::getName);                    // Returns Optional<String>

// vs map (would result in nested Optionals)
Optional<Optional<Address>> nested = empOpt.map(Employee::getAddress); // ❌ Nested!

// filter - keep value only if it matches predicate
Optional<Employee> activeEmp = empOpt
    .filter(Employee::isActive);

Optional<Employee> seniorActive = empOpt
    .filter(Employee::isActive)
    .filter(e -> e.getExperience() > 10);

// or (Java 9+) - provide alternative Optional
Optional<Employee> emp = primaryRepo.findById(id)
    .or(() -> secondaryRepo.findById(id))
    .or(() -> cacheService.findById(id));
```

11.5 Optional in Enterprise Applications

Pattern 1: Repository Layer

```
public interface EmployeeRepository extends JpaRepository<Employee, Long> {
    Optional<Employee> findByEmail(String email);
    Optional<Employee> findByEmployeeCode(String code);
}

@Service
public class EmployeeService {

    public EmployeeDTO getEmployee(Long id) {
        return employeeRepository.findById(id)
            .map(this::toDTO)
            .orElseThrow(() -> new ResourceNotFoundException("Employee", "id", id));
    }

    public EmployeeDTO getByEmailOrCode(String email, String code) {
        return employeeRepository.findByEmail(email)
            .or(() -> employeeRepository.findByEmployeeCode(code))
            .map(this::toDTO)
            .orElseThrow(() -> new ResourceNotFoundException("Employee not found"));
    }
}
```

Pattern 2: Configuration Defaults

```
public class AppConfig {

    public int getMaxRetries() {
        return Optional.ofNullable(System.getenv("MAX_RETRIES"))
            .map(Integer::parseInt)
            .filter(n -> n > 0 && n ≤ 10)
            .orElse(3);
    }

    public Duration getTimeout() {
        return Optional.ofNullable(System.getProperty("request.timeout"))
            .map(Long::parseLong)
            .map(Duration::ofMillis)
            .orElse(Duration.ofSeconds(30));
    }
}
```

Pattern 3: Chained Null-Safe Navigation

```
// Deep null-safe navigation
public String getCustomerCity(Order order) {
    return Optional.ofNullable(order)
        .map(Order::getCustomer)
        .map(Customer::getAddress)
        .map(Address::getCity)
        .map(City::getName)
        .orElse("N/A");
}

// Complex business logic
public BigDecimal calculateDiscount(Order order) {
    return Optional.ofNullable(order.getCustomer())
        .map(Customer::getLoyaltyTier)
        .map(tier -> discountStrategy.getDiscount(tier))
        .map(discount -> order.getTotal().multiply(discount))
        .orElse(BigDecimal.ZERO);
}
```

11.6 Optional Best Practices

✅ **Do's**

- Use Optional as method return type when result might be absent
- Use `orElseGet` instead of `orElse` for expensive default computations
- Use `map/flatMap` for transformations instead of `get()`
- Use `ifPresent` instead of `isPresent + get`
- Use `stream()` (Java 9+) to integrate with Stream pipelines

❌ **Don'ts**

- **Don't** use Optional as method parameter type
- **Don't** use Optional as field type (not serializable)
- **Don't** use `Optional.get()` without checking `isPresent()`
- **Don't** use Optional with collections (return empty collection instead)
- **Don't** use `Optional.of()` when value might be null
- **Don't** create Optional just to chain null checks in a single method

```java
// ❌ BAD: Optional as method parameter
public void sendEmail(Optional<String> email) { ... }

// ✅ GOOD: Use overloaded methods or nullable parameter
public void sendEmail(String email) { ... }

// ❌ BAD: Optional as field
public class Employee {
    private Optional<String> nickname; // Not serializable!
}

// ✅ GOOD: Return Optional from getter
public class Employee {
    private String nickname;
    public Optional<String> getNickname() {
        return Optional.ofNullable(nickname);
    }
}

// ❌ BAD: Returning Optional of collection
public Optional<List<Employee>> findByDept(String dept) { ... }

// ✅ GOOD: Return empty collection
public List<Employee> findByDept(String dept) {
    // Return Collections.emptyList() when no results
}

// ❌ BAD: orElse with expensive operation
Employee emp = findById(id).orElse(loadFromExternalSystem(id));
// loadFromExternalSystem() is called EVEN IF findById returns a value!

// ✅ GOOD: orElseGet for lazy evaluation
Employee emp = findById(id).orElseGet(() -> loadFromExternalSystem(id));
// loadFromExternalSystem() is called ONLY when findById returns empty
```

11.7 Optional with Streams

```
// Stream of Optionals → Stream of values
List<Optional<String>> optionals = List.of(
    Optional.of("A"), Optional.empty(), Optional.of("B"), Optional.empty()
);

// Java 8: filter + map
List<String> values = optionals.stream()
    .filter(Optional::isPresent)
    .map(Optional::get)
    .collect(Collectors.toList()); // ["A", "B"]

// Java 9+: flatMap with Optional.stream()
List<String> values = optionals.stream()
    .flatMap(Optional::stream)
    .collect(Collectors.toList()); // ["A", "B"]

// Finding first present Optional
Optional<Employee> result = Stream.of(
        cache.findById(id),
        primaryDb.findById(id),
        secondaryDb.findById(id)
    )
    .filter(Optional::isPresent)
    .map(Optional::get)
    .findFirst();
```

Chapter 12: CompletableFuture

Asynchronous programming and non-blocking computation

12.1 What is CompletableFuture?

CompletableFuture is a class that represents a future result of an asynchronous computation. Unlike the older `Future` interface, it supports combining, composing, and chaining asynchronous operations in a non-blocking, functional style. It implements both `Future` and `CompletionStage` interfaces.

12.2 Creating CompletableFuture

```
// supplyAsync - runs a Supplier asynchronously, returns a result
CompletableFuture<String> future = CompletableFuture.supplyAsync(() -> {
    // Runs in ForkJoinPool.commonPool() by default
    return externalService.fetchData();
});

// runAsync - runs a Runnable asynchronously, no return value
CompletableFuture<Void> future = CompletableFuture.runAsync(() -> {
    auditService.logAction("user-login");
});

// Custom Executor
ExecutorService executor = Executors.newFixedThreadPool(10);
CompletableFuture<String> future = CompletableFuture.supplyAsync(
    () -> apiClient.call("/users"),
    executor
);

// Already completed futures
CompletableFuture<String> completed = CompletableFuture.completedFuture("result");
CompletableFuture<String> failed = CompletableFuture.failedFuture(
    new RuntimeException("Error"));
```

12.3 Transforming Results

thenApply (map-equivalent)

```
// Transform the result when it completes
CompletableFuture<UserDTO> userFuture = CompletableFuture
    .supplyAsync(() -> userRepository.findById(userId))
    .thenApply(user -> userMapper.toDTO(user))
    .thenApply(dto -> {
        dto.setLastAccessed(LocalDateTime.now());
        return dto;
    });

// Async variant - transformation runs in separate thread
CompletableFuture<UserDTO> userFuture = CompletableFuture
    .supplyAsync(() -> userRepository.findById(userId))
    .thenApplyAsync(user -> userMapper.toDTO(user));
```

thenAccept (Consumer) and thenRun (Runnable)

```
// thenAccept - consume the result, return Void
CompletableFuture.supplyAsync(() -> orderService.getTotal(orderId))
    .thenAccept(total -> emailService.sendInvoice(customerId, total));

// thenRun - execute after completion, ignoring result
CompletableFuture.supplyAsync(() -> orderService.process(order))
    .thenRun(() -> System.out.println("Order processing complete"));
```

12.4 Composing Futures

thenCompose (flatMap-equivalent)

```
// Chain dependent async operations (avoids nested CompletableFuture)
CompletableFuture<OrderDetails> orderDetails = CompletableFuture
    .supplyAsync(() -> userService.getUser(userId))
    .thenCompose(user -> orderService.getLatestOrder(user.getId()))
    .thenCompose(order -> shippingService.getShippingDetails(order.getId()));

// Without thenCompose (nested futures - BAD)
CompletableFuture<CompletableFuture<Order>> nested =
    CompletableFuture.supplyAsync(() -> userService.getUser(userId))
        .thenApply(user -> orderService.getLatestOrder(user.getId())); // Nested!
```

thenCombine – Combine Two Independent Futures

```
// Run two futures in parallel, combine results
CompletableFuture<UserProfile> profileFuture =
    CompletableFuture.supplyAsync(() -> userService.getProfile(userId));
CompletableFuture<List<Order>> ordersFuture =
    CompletableFuture.supplyAsync(() -> orderService.getOrders(userId));

CompletableFuture<DashboardData> dashboard = profileFuture
    .thenCombine(ordersFuture, (profile, orders) ->
        new DashboardData(profile, orders));

// Three or more parallel futures
CompletableFuture<String> name = CompletableFuture.supplyAsync(() -> getName());
CompletableFuture<Integer> age = CompletableFuture.supplyAsync(() -> getAge());
CompletableFuture<String> email = CompletableFuture.supplyAsync(() -> getEmail());

CompletableFuture<UserInfo> userInfo = name
    .thenCombine(age, (n, a) -> new UserInfo(n, a))
    .thenCombine(email, (info, e) -> { info.setEmail(e); return info; });
```

12.5 Error Handling

```
// exceptionally - recover from errors
CompletableFuture<String> result = CompletableFuture
    .supplyAsync(() -> riskyOperation())
    .exceptionally(ex -> {
        logger.error("Operation failed", ex);
        return "default-value";
    });

// handle - handle both success and failure
CompletableFuture<Response> response = CompletableFuture
    .supplyAsync(() -> apiClient.call(request))
    .handle((result, ex) -> {
        if (ex != null) {
            return Response.error(ex.getMessage());
        }
        return Response.success(result);
    });

// whenComplete - observe result/error without modifying
CompletableFuture<String> future = CompletableFuture
    .supplyAsync(() -> processData())
    .whenComplete((result, ex) -> {
        if (ex != null) {
            metrics.incrementFailure();
            logger.error("Processing failed", ex);
        } else {
            metrics.incrementSuccess();
            logger.info("Processing succeeded: {}", result);
        }
    });
```

12.6 Combining Multiple Futures

```
// allOf - wait for ALL futures to complete
List<CompletableFuture<String>> futures = urls.stream()
    .map(url -> CompletableFuture.supplyAsync(() -> httpClient.get(url)))
    .collect(Collectors.toList());

CompletableFuture<Void> allDone = CompletableFuture.allOf(
    futures.toArray(new CompletableFuture[0])
);

// Get all results after allOf completes
CompletableFuture<List<String>> allResults = allDone.thenApply(v ->
    futures.stream()
        .map(CompletableFuture::join)
        .collect(Collectors.toList())
);

// anyOf - complete when ANY future completes
CompletableFuture<Object> fastest = CompletableFuture.anyOf(
    CompletableFuture.supplyAsync(() -> callPrimaryApi()),
    CompletableFuture.supplyAsync(() -> callSecondaryApi()),
    CompletableFuture.supplyAsync(() -> callCacheApi())
);
```

12.7 Enterprise Patterns

Pattern: Parallel API Calls in Microservice

```
@Service
public class DashboardService {
    @Autowired private UserServiceClient userClient;
    @Autowired private OrderServiceClient orderClient;
    @Autowired private NotificationServiceClient notifClient;

    public CompletableFuture<DashboardDTO> getDashboard(Long userId) {
        CompletableFuture<UserDTO> userFuture =
            CompletableFuture.supplyAsync(() -> userClient.getUser(userId));
        CompletableFuture<List<OrderDTO>> ordersFuture =
            CompletableFuture.supplyAsync(() -> orderClient.getOrders(userId));
        CompletableFuture<List<NotificationDTO>> notifsFuture =
            CompletableFuture.supplyAsync(() -> notifClient.getNotifications(userId))

        return CompletableFuture.allOf(userFuture, ordersFuture, notifsFuture)
            .thenApply(v -> new DashboardDTO(
                userFuture.join(),
                ordersFuture.join(),
                notifsFuture.join()
            ))
            .exceptionally(ex -> {
                log.error("Dashboard aggregation failed", ex);
                return DashboardDTO.empty();
            });
    }
}
```

Pattern: Timeout and Fallback

```
// Java 9+ orTimeout and completeOnTimeout
CompletableFuture<Response> response = CompletableFuture
    .supplyAsync(() -> slowService.call())
    .orTimeout(5, TimeUnit.SECONDS)                    // Fails after 5s
    .exceptionally(ex -> Response.timeout());

// Java 8 manual timeout
public static <T> CompletableFuture<T> withTimeout(
        CompletableFuture<T> future, Duration timeout, T fallback) {
    CompletableFuture<T> timeoutFuture = new CompletableFuture<>();
    ScheduledExecutorService scheduler = Executors.newScheduledThreadPool(1);
    scheduler.schedule(
        () -> timeoutFuture.complete(fallback),
        timeout.toMillis(), TimeUnit.MILLISECONDS
    );
    return CompletableFuture.anyOf(future, timeoutFuture)
        .thenApply(result -> (T) result);
}
```

⚠️ **CompletableFuture Common Mistakes**

- **Not handling errors** – Always add `exceptionally()` or `handle()`
- **Blocking with get()** – Avoid `.get()` in reactive code; use callback chains
- **Using common ForkJoinPool** for I/O – Use custom executor for blocking operations
- **Creating too many threads** – Use bounded thread pools for production

Chapter 13: Parallel Streams

Leveraging multi-core processors for data processing

13.1 What are Parallel Streams?

Parallel streams split the source data into multiple chunks, process each chunk on separate threads using the ForkJoinPool, and then combine the results. They provide transparent parallelism for stream operations without explicit thread management.

13.2 Creating Parallel Streams

```
// From collection
Stream<Employee> parallel1 = employees.parallelStream();

// From existing stream
Stream<Employee> parallel2 = employees.stream().parallel();

// Check if parallel
boolean isParallel = parallel1.isParallel(); // true

// Convert back to sequential
Stream<Employee> sequential = parallel1.sequential();
```

13.3 How Parallel Streams Work Internally

```
ForkJoinPool Work-Stealing Algorithm Source Data: [1, 2, 3, 4, 5, 6, 7, 8] |
┌──────┴──────┐ | SPLIT | (Spliterator.trySplit()) [1,2,3,4] [5,6,7,8] | | ┌──────┴──────┐
┌──────┴──────┐ | SPLIT | | SPLIT | [1,2] [3,4] [5,6] [7,8] | | | | Thread1 Thread2
Thread3 Thread4 | | | | ▼ ▼ ▼ ▼ [sum=3] [sum=7] [sum=11] [sum=15] | | ┌──────┴──────┐
┌──────┴──────┐ | COMBINE | | COMBINE | [sum=10] [sum=26] | | └──────┬──────────────┘ |
COMBINE [sum=36]
```

13.4 When to Use Parallel Streams

Criteria	Use Parallel	Use Sequential
Data size	> 10,000 elements	< 1,000 elements
Computation per element	CPU-intensive (heavy computation)	Simple operations (getter/setter)
Source data structure	ArrayList, arrays (good Spliterator)	LinkedList, TreeSet (poor splitting)
Operation type	Stateless, independent operations	Stateful, order-dependent operations
Side effects	No shared mutable state	Operations with side effects
I/O operations	No (thread pool starved)	Yes (use CompletableFuture instead)

13.5 Safe Parallel Stream Examples

```
// ✅ Good: CPU-intensive, large dataset, no shared state
double totalRisk = portfolios.parallelStream()
    .mapToDouble(this::computeRiskScore)  // CPU-intensive
    .sum();

// ✅ Good: Independent processing, no side effects
List<ImageResult> thumbnails = images.parallelStream()
    .map(this::generateThumbnail)  // CPU-intensive image processing
    .collect(Collectors.toList());

// ✅ Good: Large dataset reduction
long count = transactions.parallelStream()
    .filter(t -> t.getAmount() > 10000)
    .count();
```

13.6 Parallel Stream Pitfalls

```
// ❌ BAD: Shared mutable state
List<Integer> result = new ArrayList<>();  // NOT thread-safe!
numbers.parallelStream()
    .filter(n -> n > 5)
    .forEach(result::add);  // Race condition! Lost updates!

// ✅ FIX: Use collect
List<Integer> result = numbers.parallelStream()
    .filter(n -> n > 5)
    .collect(Collectors.toList());

// ❌ BAD: Order-dependent operations
List<String> result = names.parallelStream()
    .reduce("", (a, b) -> a + b);  // Wrong order in parallel!

// ❌ BAD: I/O operations (starves ForkJoinPool)
list.parallelStream()
    .forEach(item -> {
        httpClient.post(item);  // Blocks ForkJoinPool threads
    });

// ✅ FIX: Use CompletableFuture with custom executor
ExecutorService executor = Executors.newFixedThreadPool(20);
List<CompletableFuture<Void>> futures = list.stream()
    .map(item -> CompletableFuture.runAsync(
        () -> httpClient.post(item), executor))
    .collect(Collectors.toList());
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
```

13.7 Custom ForkJoinPool

```
// Default: ForkJoinPool.commonPool() (shared across all parallel streams!)
// Custom ForkJoinPool for isolation
ForkJoinPool customPool = new ForkJoinPool(8);  // 8 threads
List<Result> results = customPool.submit(() ->
    largeDataset.parallelStream()
        .map(this::heavyComputation)
        .collect(Collectors.toList())
).get();  // Execute in custom pool

// Always shut down when done
customPool.shutdown();
```

🚨 **Critical: ForkJoinPool Starvation**

All parallel streams in your application share the `ForkJoinPool.commonPool()` by default. If one parallel stream has blocking I/O operations, it can starve the entire application's parallel stream processing. Always use a dedicated ForkJoinPool for heavy or blocking operations, and prefer `CompletableFuture` with custom executors for I/O-bound work.

13.8 Performance Comparison Guide

```
// Benchmark template (not micro-benchmark quality, illustrative)
long start = System.nanoTime();
List<Result> sequential = data.stream()
    .map(this::heavyComputation)
    .collect(Collectors.toList());
long seqTime = System.nanoTime() - start;

start = System.nanoTime();
List<Result> parallel = data.parallelStream()
    .map(this::heavyComputation)
    .collect(Collectors.toList());
long parTime = System.nanoTime() - start;

// Rules of thumb:
// - Parallel is NOT always faster
// - Thread creation/management has overhead
// - Small datasets: sequential is often 10x faster
// - Only use parallel when measured improvement justifies complexity
```

Chapter 14: Default & Static Methods in Interfaces

Evolving interfaces without breaking existing implementations

14.1 Why Default Methods?

Before Java 8, adding a new method to an interface would break ALL implementing classes. Default methods solve this by allowing interfaces to have method implementations. This enabled the addition of `stream()`, `forEach()`, and other methods to the Collections API without breaking backward compatibility.

14.2 Default Methods

```
public interface PaymentProcessor {
    // Abstract method - must be implemented
    PaymentResult process(Payment payment);

    // Default method - has implementation, can be overridden
    default PaymentResult processWithRetry(Payment payment, int maxRetries) {
        for (int i = 0; i < maxRetries; i++) {
            try {
                return process(payment);
            } catch (Exception e) {
                if (i == maxRetries - 1) throw e;
                try { Thread.sleep(1000 * (i + 1)); }
                catch (InterruptedException ie) { Thread.currentThread().interrupt();
            }
        }
        throw new RuntimeException("Unreachable");
    }

    default boolean supports(PaymentMethod method) {
        return true; // Default: supports all methods
    }
}

// Implementation can override default methods
public class CreditCardProcessor implements PaymentProcessor {
    @Override
    public PaymentResult process(Payment payment) {
        return gateway.charge(payment);
    }

    @Override
    public boolean supports(PaymentMethod method) {
        return method == PaymentMethod.CREDIT_CARD
            || method == PaymentMethod.DEBIT_CARD;
    }
    // processWithRetry uses default implementation
}
```

14.3 Static Methods in Interfaces

```
public interface Validator<T> {
    boolean validate(T value);

    // Static factory methods
    static <T> Validator<T> notNull() {
        return value -> value != null;
    }

    static Validator<String> notBlank() {
        return value -> value != null && !value.trim().isEmpty();
    }

    static Validator<String> maxLength(int max) {
        return value -> value != null && value.length() <= max;
    }

    static <T> Validator<T> combine(List<Validator<T>> validators) {
        return value -> validators.stream().allMatch(v -> v.validate(value));
    }
}

// Usage
Validator<String> emailValidator = Validator.combine(List.of(
    Validator.notBlank(),
    Validator.maxLength(255),
    email -> email.contains("@")
));
```

14.4 The Diamond Problem

```
interface A {
    default String greet() { return "Hello from A"; }
}

interface B {
    default String greet() { return "Hello from B"; }
}

// ❌ COMPILE ERROR: Inherits greet() from both A and B
class C implements A, B { }

// ✅ FIX: Must override and resolve explicitly
class C implements A, B {
    @Override
    public String greet() {
        return A.super.greet(); // Choose A's implementation
        // Or: return B.super.greet();
        // Or: return "Hello from C"; // Own implementation
    }
}

// Resolution Rules:
// 1. Class wins over interface (if class provides implementation)
// 2. Sub-interface wins over super-interface (most specific wins)
// 3. If ambiguous, class MUST override and resolve explicitly
```

14.5 Real-World Usage in Spring Framework

```
// Spring Data JPA uses default methods extensively
public interface UserRepository extends JpaRepository<User, Long> {

    Optional<User> findByEmail(String email);

    // Custom default method combining JPA methods
    default User findByEmailOrThrow(String email) {
        return findByEmail(email)
            .orElseThrow(() -> new UserNotFoundException(email));
    }

    default List<User> findActiveUsers() {
        return findAll().stream()
            .filter(User::isActive)
            .sorted(Comparator.comparing(User::getCreatedAt).reversed())
            .collect(Collectors.toList());
    }
}
```

Architecture Insight

Default methods enable the **Interface Segregation Principle** (ISP) more effectively. Instead of large interfaces with many methods, you can have smaller interfaces with sensible defaults, allowing implementors to override only what they need. This is the foundation of Spring's functional endpoints and reactive programming support.

Chapter 15: Interview Questions – Beginner & Intermediate

50+ questions to solidify your Java 8 foundation

15.1 Beginner Level Questions

Q1. What are the main features introduced in Java 8?

> Key features include: Lambda expressions, Functional interfaces, Stream API, Optional class, Method references, Default and static methods in interfaces, new Date/Time API (java.time), CompletableFuture, Nashorn JavaScript engine, and the java.util.function package with built-in functional interfaces like Predicate, Function, Consumer, and Supplier.

Q2. What is a lambda expression?

> A lambda expression is an anonymous function that provides a concise way to implement a functional interface. It has no name, no access modifier, and no return type declaration. Syntax: `(parameters) → expression` or `(parameters) → { statements; }`. Example: `(a, b) → a + b` implements a BiFunction that adds two numbers.

Q3. What is a functional interface?

> A functional interface is an interface with exactly one abstract method (SAM - Single Abstract Method). It can have any number of default or static methods. Examples: Runnable, Callable, Comparator, Predicate, Function, Consumer, Supplier. The `@FunctionalInterface` annotation is optional but recommended for compile-time validation.

Q4. What is the difference between Predicate, Function, Consumer, and Supplier?

Interface	Input	Output	Method	Use Case
Predicate<T>	T	boolean	test(T)	Filtering, conditions
Function<T,R>	T	R	apply(T)	Transformation
Consumer<T>	T	void	accept(T)	Side effects (print, save)
Supplier<T>	none	T	get()	Factory, lazy values

Q5. What are the four types of method references?

1) **Static method:** `Integer::parseInt` → `s -> Integer.parseInt(s)`
2) **Instance method of specific object:** `System.out::println` → `x -> System.out.println(x)`
3) **Instance method of arbitrary object:** `String::toUpperCase` → `s -> s.toUpperCase()`
4) **Constructor:** `ArrayList::new` → `() -> new ArrayList()`

Q6. What is a Stream in Java 8?

A Stream is a sequence of elements that supports sequential and parallel aggregate operations. Streams don't store data—they convey elements from a data source (collection, array, generator) through a pipeline of operations. Streams are lazy (intermediate operations only execute when a terminal operation is called) and can be consumed only once.

Q7. What is the difference between intermediate and terminal operations?

Intermediate operations return a new Stream and are lazy (not executed until terminal operation). Examples: filter, map, flatMap, sorted, distinct, limit, skip, peek.

Terminal operations produce a result or side effect and trigger pipeline execution. Examples: collect, reduce, forEach, count, min, max, findFirst, findAny, anyMatch, allMatch, noneMatch.

Q8. What is Optional and why was it introduced?

Optional is a container object that may or may not contain a non-null value. It was introduced to reduce NullPointerExceptions by making the possibility of absence explicit in the type system. Instead of returning null, methods return `Optional.empty()`, forcing callers to handle the absent case. Key methods: `of()`, `ofNullable()`, `isPresent()`, `get()`, `orElse()`, `orElseGet()`, `orElseThrow()`, `map()`, `flatMap()`, `filter()`.

Q9. What is the difference between Collection and Stream?

> **Collection:** Stores elements in memory, external iteration, can be iterated multiple times, eagerly computed, finite, supports add/remove.
> **Stream:** Doesn't store elements, internal iteration, can be consumed only once, lazily computed, can be infinite, read-only (doesn't modify source). Streams are for computation; collections are for data storage.

Q10. What is lazy evaluation in Streams?

> Lazy evaluation means intermediate operations are not executed until a terminal operation is invoked. This enables optimizations like loop fusion (combining operations into a single pass) and short-circuiting (stopping early when possible). For example, `stream.filter(x → x > 5).findFirst()` stops processing as soon as it finds the first element greater than 5, not after filtering ALL elements.

Q11. What is the difference between `map()` and `flatMap()`?

> `map()` transforms each element into exactly one result (1-to-1). `flatMap()` transforms each element into a Stream and flattens all streams into one (1-to-many). Use `flatMap` when each element can produce zero or more results, such as flattening nested collections.

```
// map: Stream<List<String>> (nested)
Stream<List<String>> mapped = lists.stream().map(list → list);

// flatMap: Stream<String> (flattened)
Stream<String> flatMapped = lists.stream().flatMap(Collection::stream);
```

Q12. What is `effectively final` in Java 8?

> A local variable is effectively final if its value is never changed after initialization, even without the `final` keyword. Lambda expressions can only capture local variables that are effectively final. This restriction prevents race conditions when lambdas are executed in different threads (e.g., parallel streams).

Q13. What is the difference between `findFirst()` and `findAny()`?

> `findFirst()` returns the first element in encounter order. `findAny()` returns any element without order guarantee. In sequential streams, both behave the same. In parallel streams, `findAny()` is faster because it can return the first element found by any thread, while `findFirst()` must coordinate threads to ensure order.

Q14. What is the default method in an interface? Why was it introduced?

> A default method is a method in an interface that has an implementation (body). It was introduced to allow evolution of interfaces without breaking existing implementations. This enabled adding methods like `stream()`, `forEach()`, `sort()` to the Collection interface without forcing all existing implementations to update.

Q15. Explain `orElse` vs `orElseGet` in Optional.

> `orElse(T value)` always evaluates the default value, even when Optional has a value. `orElseGet(Supplier)` evaluates the supplier only when Optional is empty (lazy). Use `orElseGet` when the default involves computation or object creation.

```
// orElse - createDefault() is ALWAYS called
Employee e = opt.orElse(createDefault());

// orElseGet - createDefault() only called when opt is empty
Employee e = opt.orElseGet(() -> createDefault());
```

15.2 Intermediate Level Questions

Q16. Explain the `reduce()` operation with examples.

> Reduce combines all elements of a stream into a single result using an associative accumulation function. It has three forms:

```
// 1. With identity: T reduce(T identity, BinaryOperator<T>)
int sum = numbers.stream().reduce(0, Integer::sum); // returns int

// 2. Without identity: Optional<T> reduce(BinaryOperator<T>)
Optional<Integer> sum = numbers.stream().reduce(Integer::sum);

// 3. With identity, accumulator, combiner (for parallel):
int total = strings.stream().reduce(0,
    (sum, str) -> sum + str.length(),  // accumulator
    Integer::sum);                      // combiner
```

Q17. Difference between `reduce()` and `collect()`?

> `reduce()` produces an immutable result by repeatedly combining two values into one (folding). `collect()` accumulates elements into a mutable container (like ArrayList, HashMap). Use `reduce` for computing aggregate values (sum, max, concatenation); use `collect` for building collections or complex data structures. `collect` is more efficient for mutable reductions because it avoids creating intermediate objects.

Q18. How does `groupingBy` work? Give examples.

```
// Simple grouping
Map<String, List<Employee>> byDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment));

// Grouping with count
Map<String, Long> countByDept = employees.stream()
    .collect(Collectors.groupingBy(Employee::getDepartment,
             Collectors.counting()));

// Multi-level grouping
Map<String, Map<String, List<Employee>>> byDeptAndRole =
    employees.stream().collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.groupingBy(e -> e.getRole().name())));
```

Q19. What is the Diamond Problem with default methods?

When a class implements two interfaces that both have a default method with the same signature, there's ambiguity about which implementation to use. Java forces the class to override the method and resolve the conflict explicitly using `InterfaceName.super.method()` syntax. Resolution rules: 1) Class wins over interface. 2) More specific interface wins. 3) If still ambiguous, class must override.

Q20. What is `Spliterator` and how does it relate to streams?

Spliterator (Splittable Iterator) is the internal mechanism that enables both sequential and parallel stream processing. It provides: `tryAdvance()` to process one element, `trySplit()` to divide the data source for parallel processing, `estimateSize()` for remaining elements, and `characteristics()` for properties like ORDERED, SORTED, DISTINCT. When you call `parallelStream()`, the Spliterator repeatedly splits the data into halves for Fork/Join processing.

Q21. When should you NOT use parallel streams?

Avoid parallel streams when: 1) Dataset is small (<1000 elements). 2) Operations involve I/O (blocks ForkJoinPool threads). 3) Operations have side effects or shared mutable state. 4) Source is LinkedList or other poorly-splittable structures. 5) Order matters and operations are stateful. 6) The computation per element is trivial. 7) You're already inside a shared ForkJoinPool context. Use benchmarking to verify parallel actually improves performance.

Q22. Explain `CompletableFuture` and how it differs from `Future`.

> `Future` (Java 5) is limited: can only check if done (`isDone()`), block to get result (`get()`), or cancel. No chaining, no combining, no callbacks. `CompletableFuture` (Java 8) adds: non-blocking callbacks (`thenApply` , `thenAccept`), composition (`thenCompose` , `thenCombine`), error handling (`exceptionally` , `handle`), combining multiple futures (`allOf` , `anyOf`), and async execution with custom executors.

Q23. What is `thenApply` vs `thenCompose` in CompletableFuture?

> `thenApply` is like `map` — transforms the result synchronously. `thenCompose` is like `flatMap` — chains two asynchronous operations sequentially, avoiding nested CompletableFutures.

```
// thenApply: Function<T, R> → CompletableFuture<R>
future.thenApply(result -> result.toUpperCase());

// thenCompose: Function<T, CompletableFuture<R>> → CompletableFuture<R>
future.thenCompose(userId -> fetchUserOrders(userId));
```

Q24. How do you chain multiple Predicate conditions?

```
Predicate<Employee> isActive = Employee::isActive;
Predicate<Employee> isSenior = e -> e.getExperience() > 10;
Predicate<Employee> highSalary = e -> e.getSalary() > 100000;

// AND
Predicate<Employee> combined = isActive.and(isSenior).and(highSalary);

// OR
Predicate<Employee> anyCondition = isActive.or(isSenior);

// NEGATE
Predicate<Employee> inactive = isActive.negate();

// Usage
List<Employee> eligible = employees.stream()
    .filter(combined)
    .collect(Collectors.toList());
```

Q25. What are stateless vs stateful operations in streams?

> **Stateless:** Each element can be processed independently without knowledge of other elements. Examples: filter, map, flatMap, peek. These work well with parallel streams.
>
> **Stateful:** Processing an element may depend on other elements. Examples: distinct (tracks seen elements), sorted (needs all elements), limit/skip (needs count). Stateful operations may need to buffer elements and can be less efficient in parallel.

Q26. Explain `Collectors.toMap()` and how to handle duplicate keys.

```
// Basic: throws IllegalStateException on duplicate keys!
Map<Long, Employee> map = emps.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));

// With merge function: handles duplicates
Map<String, Employee> map = emps.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        (e1, e2) -> e1.getSalary() > e2.getSalary() ? e1 : e2));

// With map type specification
TreeMap<Long, String> sorted = emps.stream()
    .collect(Collectors.toMap(
        Employee::getId, Employee::getName,
        (v1, v2) -> v1, TreeMap::new));
```

Q27. How does stream pipeline optimization work?

Stream pipelines are optimized through: 1) **Lazy evaluation:** No work until terminal operation. 2) **Loop fusion:** Multiple operations merged into single pass. 3) **Short-circuiting:** Operations like findFirst, limit, anyMatch stop early. 4) **Spliterator characteristics:** Operations like distinct() on a DISTINCT source become no-ops. 5) **Sized characteristics:** If source size is known, operations like toArray() can pre-allocate.

Q28. What is Function.compose() vs Function.andThen()?

```
Function<Integer, Integer> doubleIt = x -> x * 2;
Function<Integer, Integer> addThree = x -> x + 3;

// andThen: f.andThen(g) = g(f(x)) — applies f first, then g
int result1 = doubleIt.andThen(addThree).apply(5);
// (5 * 2) + 3 = 13

// compose: f.compose(g) = f(g(x)) — applies g first, then f
int result2 = doubleIt.compose(addThree).apply(5);
// (5 + 3) * 2 = 16
```

Q29. Can we use `this` inside a lambda? How does it differ from anonymous class?

Yes, but `this` in a lambda refers to the **enclosing class instance**, not the lambda itself (lambdas don't create their own scope). In an anonymous inner class, `this` refers to the **anonymous class instance**. This difference exists because lambdas are not compiled as separate classes—they become private methods of the enclosing class via invokedynamic.

Q30. What are the performance implications of autoboxing in streams?

> When using `Stream<Integer>` instead of `IntStream`, every primitive int is autoboxed to Integer objects. This causes: extra object creation, increased garbage collection, cache misses (objects not contiguous in memory). For large datasets, always use `IntStream`, `LongStream`, `DoubleStream` or operations like `mapToInt()`, `mapToDouble()` to avoid boxing overhead.

Chapter 16: Advanced & Architect-Level Questions

70+ questions for senior developer and architect interviews

16.1 Advanced Java 8 Questions

Q31. How are lambda expressions compiled internally? How is this different from anonymous classes?

> Lambda expressions are compiled using `invokedynamic` bytecode instruction (introduced in Java 7). The compiler: 1) Converts the lambda body into a private static/instance method in the enclosing class. 2) Generates an `invokedynamic` call site that, at runtime, uses `LambdaMetafactory` to generate an implementing class. Unlike anonymous classes: no separate .class files, no forced object creation on every invocation (JVM can cache instances), better JIT optimization. A non-capturing lambda (no external variables) can be instantiated as a singleton.

Q32. Explain the internal architecture of Stream API.

> Internally, a stream pipeline consists of: 1) **Source** (Spliterator): provides elements and splitting capabilities. 2) **AbstractPipeline** chain: each intermediate operation creates a new pipeline stage wrapping the previous one. 3) Operations are represented as **Sink** instances that form a chain (like a linked list). 4) When terminal operation triggers, it calls `wrapSink()` to create the operation chain, then iterates through the source Spliterator. For parallel: the Spliterator splits the source recursively via `trySplit()`, submitting each half to the ForkJoinPool.

Q33. What is the ForkJoinPool and how do parallel streams use it?

> ForkJoinPool is an ExecutorService that uses a work-stealing algorithm. Each worker thread has its own deque of tasks. When a thread finishes its work, it "steals" tasks from other threads' deques. Parallel streams use `ForkJoinPool.commonPool()` by default (size = Runtime.availableProcessors() - 1). The Spliterator recursively splits data, each split becomes a ForkJoinTask. Results are combined bottom-up. **Warning:** All parallel streams share the common pool. Blocking operations can starve others.

Q34. How would you implement a custom Collector?

```
// Custom collector: Collect to ImmutableList
Collector<String, List<String>, List<String>> toImmutableList =
    Collector.of(
        ArrayList::new,                // supplier
        List::add,                      // accumulator
        (list1, list2) -> {             // combiner (for parallel)
            list1.addAll(list2);
            return list1;
        },
        Collections::unmodifiableList, // finisher
        Collector.Characteristics.UNORDERED
    );

// Usage
List<String> immutable = stream.collect(toImmutableList);
```

Q35. Explain Collector characteristics: CONCURRENT, UNORDERED, IDENTITY_FINISH.

CONCURRENT: Accumulator function can be called concurrently from multiple threads on the same result container (e.g., ConcurrentHashMap).

UNORDERED: Collection operation does not need to preserve encounter order. Enables optimizations in parallel streams.

IDENTITY_FINISH: The finisher function is the identity function (no post-processing needed). The accumulator result IS the final result. This allows the framework to skip the finisher step and cast directly.

Q36. How do you handle checked exceptions in lambda expressions?

```
// Problem: Functional interfaces don't declare checked exceptions
// stream.map(url -> new URL(url)) // ❌ Compile error: MalformedURLException

// Solution 1: Wrapper method
public static <T, R> Function<T, R> unchecked(CheckedFunction<T, R> f) {
    return t -> {
        try { return f.apply(t); }
        catch (Exception e) { throw new RuntimeException(e); }
    };
}
stream.map(unchecked(url -> new URL(url)));

// Solution 2: Try-catch inside lambda
stream.map(url -> {
    try { return new URL(url); }
    catch (MalformedURLException e) { return null; }
}).filter(Objects::nonNull);

// Solution 3: Custom functional interface
@FunctionalInterface
interface CheckedFunction<T, R> {
    R apply(T t) throws Exception;
}
```

Q37. What is the Spliterator.characteristics() and why does it matter?

Characteristics describe properties of the data source and enable stream optimizations: `ORDERED` (elements have defined encounter order, e.g., List), `DISTINCT` (all elements are unique, e.g., Set), `SORTED` (elements are sorted), `SIZED` (size is known in advance), `NONNULL` (no null elements), `IMMUTABLE` (source won't be modified), `CONCURRENT` (can be modified concurrently), `SUBSIZED` (all splits are sized). For example, calling `distinct()` on a DISTINCT source is a no-op.

Q38. How does `Stream.iterate()` work internally and how to create bounded iterate in Java 8?

```
// Java 8: Infinite, must use limit()
Stream.iterate(0, n -> n + 2).limit(10); // First 10 even numbers

// Java 9+: Bounded iterate with predicate
Stream.iterate(0, n -> n < 20, n -> n + 2); // Even numbers < 20

// Internal: Uses a Spliterator that generates elements lazily
// Each call to tryAdvance() applies the UnaryOperator to previous value
```

Q39. How does `Collectors.groupingByConcurrent()` differ from `groupingBy()`?

> `groupingByConcurrent()` uses a ConcurrentHashMap and allows concurrent accumulation from multiple threads. It's designed for parallel streams. It has CONCURRENT and UNORDERED characteristics, meaning it doesn't preserve encounter order. Use it when order doesn't matter and you're processing large data sets in parallel for better performance.

Q40. How would you implement a sliding window operation with streams?

> ```
> public static <T> Stream<List<T>> sliding(List<T> list, int windowSize) {
>  return IntStream.rangeClosed(0, list.size() - windowSize)
>  .mapToObj(i -> list.subList(i, i + windowSize));
> }
>
> // Usage: Moving average
> List<Integer> data = List.of(1, 2, 3, 4, 5, 6, 7);
> List<Double> movingAvg = sliding(data, 3)
>  .map(window -> window.stream().mapToInt(Integer::intValue).average().orElse(0))
>  .collect(Collectors.toList());
> // [2.0, 3.0, 4.0, 5.0, 6.0]
> ```

16.2 Architect-Level Questions

Q41. When designing a high-throughput data processing pipeline, how do you choose between Streams, CompletableFuture, and reactive frameworks (RxJava/Reactor)?

> **Streams:** Best for synchronous, in-memory, CPU-bound data transformation. Single pipeline, no backpressure.
>
> **CompletableFuture:** Best for orchestrating few independent async I/O operations (API calls). Manual composition, no backpressure.
>
> **Reactive (RxJava/Reactor):** Best for complex async event-driven systems with backpressure, retry logic, time-based operations. Higher learning curve but necessary for high-throughput I/O-bound systems with many concurrent connections.
>
> **Decision framework:** CPU-bound + in-memory → Streams. Few async I/O calls → CompletableFuture. Many concurrent I/O + backpressure → Reactive.

Q42. How would you design a thread-safe caching mechanism using Java 8 features?

```
public class CacheService<K, V> {
    private final ConcurrentHashMap<K, V> cache = new ConcurrentHashMap<>();

    // computeIfAbsent: atomic check-and-compute
    public V get(K key, Function<K, V> loader) {
        return cache.computeIfAbsent(key, loader);
    }

    // With TTL using CompletableFuture
    private final ConcurrentHashMap<K, CompletableFuture<V>> asyncCache =
        new ConcurrentHashMap<>();

    public CompletableFuture<V> getAsync(K key, Function<K, V> loader) {
        return asyncCache.computeIfAbsent(key, k ->
            CompletableFuture.supplyAsync(() -> loader.apply(k)));
    }
}
```

Q43. How do you handle error aggregation in a Stream pipeline that processes batch operations?

```
// Use Either/Result pattern
record Result<T>(T value, Exception error, boolean success) {
    static <T> Result<T> ok(T value) { return new Result<>(value, null, true); }
    static <T> Result<T> fail(Exception e) { return new Result<>(null, e, false); }
}

List<Result<ProcessedOrder>> results = orders.stream()
    .map(order -> {
        try { return Result.ok(processOrder(order)); }
        catch (Exception e) { return Result.fail(e); }
    })
    .collect(Collectors.toList());

// Partition successes and failures
Map<Boolean, List<Result<ProcessedOrder>>> partitioned = results.stream()
    .collect(Collectors.partitioningBy(Result::success));

List<ProcessedOrder> successes = partitioned.get(true).stream()
    .map(Result::value).collect(Collectors.toList());
List<Exception> errors = partitioned.get(false).stream()
    .map(Result::error).collect(Collectors.toList());
```

Q44. How would you implement the Strategy Pattern using Java 8 functional interfaces?

```
// Traditional Strategy Pattern requires class for each strategy
// Java 8: Strategies are just functions

@Service
public class NotificationService {
    private final Map<NotificationType, Consumer<Notification>> strategies = Map.of
        NotificationType.EMAIL, this::sendEmail,
        NotificationType.SMS, this::sendSms,
        NotificationType.PUSH, this::sendPush,
        NotificationType.SLACK, this::sendSlack
    );

    public void send(Notification notification) {
        strategies.getOrDefault(
            notification.getType(),
            n -> log.warn("Unknown notification type: {}", n.getType())
        ).accept(notification);
    }

    // Each strategy is just a method
    private void sendEmail(Notification n) { /* ... */ }
    private void sendSms(Notification n) { /* ... */ }
    // ...
}
```

Q45. Explain how you would build a validation framework using Predicate chaining.

```
public class ValidationRule<T> {
    private final Predicate<T> predicate;
    private final String errorMessage;

    public static <T> ValidationRule<T> of(Predicate<T> predicate, String msg) {
        return new ValidationRule<>(predicate, msg);
    }
}

public class Validator<T> {
    private final List<ValidationRule<T>> rules = new ArrayList<>();

    public Validator<T> addRule(Predicate<T> predicate, String msg) {
        rules.add(ValidationRule.of(predicate, msg));
        return this;
    }

    public List<String> validate(T object) {
        return rules.stream()
            .filter(rule -> !rule.getPredicate().test(object))
            .map(ValidationRule::getErrorMessage)
            .collect(Collectors.toList());
    }
}

// Usage
Validator<Employee> employeeValidator = new Validator<Employee>()
    .addRule(e -> e.getName() != null, "Name is required")
    .addRule(e -> e.getEmail().contains("@"), "Invalid email")
    .addRule(e -> e.getSalary() > 0, "Salary must be positive")
    .addRule(e -> e.getAge() >= 18, "Must be 18 or older");

List<String> errors = employeeValidator.validate(employee);
```

Q46. How does Java 8's Optional handle the monad laws? Is Optional a true monad?

Optional partially follows monad patterns but is NOT a true monad because it violates the left identity law when the value is null:

1) **Left identity:** `Optional.of(x).flatMap(f)` should equal `f(x)` — this holds for non-null x but `Optional.of(null)` throws NPE.

2) **Right identity:** `m.flatMap(Optional::of)` should equal `m` — this holds.

3) **Associativity:** `m.flatMap(f).flatMap(g)` should equal `m.flatMap(x -> f(x).flatMap(g))` — this holds.

Despite not being a perfect monad, Optional's map/flatMap pattern is invaluable for null-safe value chains.

Q47. How do you design APIs that work well with streams?

1) Return `Stream<T>` from query methods for large/lazy datasets. 2) Accept functional interfaces as parameters (Predicate for filtering, Function for transformation). 3) Return `Optional<T>` for methods that might not find a result. 4) Design methods to be side-effect free. 5) Provide primitive stream variants (IntStream, etc.) for numeric operations. 6) Use immutable data objects for thread-safe streaming. 7) Support custom Spliterators for complex data sources.

Q48. What patterns exist for handling exceptions in CompletableFuture chains?

```
// Pattern 1: Recover with fallback
future.exceptionally(ex -> defaultValue);

// Pattern 2: Transform errors to domain exceptions
future.handle((result, ex) -> {
    if (ex != null) throw new DomainException("Failed", ex);
    return result;
});

// Pattern 3: Retry
public <T> CompletableFuture<T> retry(Supplier<CompletableFuture<T>> task,
                                      int attempts) {
    CompletableFuture<T> future = task.get();
    for (int i = 1; i < attempts; i++) {
        future = future.handle((result, ex) -> {
            if (ex == null) return CompletableFuture.completedFuture(result);
            return task.get();
        }).thenCompose(Function.identity());
    }
    return future;
}

// Pattern 4: Circuit breaker
future.handle((result, ex) -> {
    if (ex != null) {
        circuitBreaker.recordFailure();
        return fallback;
    }
    circuitBreaker.recordSuccess();
    return result;
});
```

Q49. How would you implement a functional pipeline pattern for request processing (like middleware)?

```
@FunctionalInterface
interface Handler<I, O> {
    O handle(I input);

    default <R> Handler<I, R> andThen(Handler<O, R> next) {
        return input -> next.handle(this.handle(input));
    }
}

// Build middleware pipeline
Handler<HttpRequest, HttpRequest> auth = request -> {
    validateToken(request.getHeader("Authorization"));
    return request;
};
Handler<HttpRequest, HttpRequest> log = request -> {
    logger.info("Processing: {}", request.getPath());
    return request;
};
Handler<HttpRequest, HttpRequest> rateLimit = request -> {
    rateLimiter.acquire();
    return request;
};
Handler<HttpRequest, HttpResponse> process = request -> {
    return controller.handle(request);
};

// Compose pipeline
Handler<HttpRequest, HttpResponse> pipeline =
    auth.andThen(log).andThen(rateLimit).andThen(process);
```

Q50. In a microservices architecture, how do you use CompletableFuture for service aggregation while handling partial failures?

> Use the **Scatter-Gather** pattern: fire multiple service calls in parallel, collect results with timeouts, handle partial failures gracefully.

```
public DashboardDTO buildDashboard(String userId) {
    CompletableFuture<Optional<UserDTO>> user = callWithFallback(
        () → userService.getUser(userId), Duration.ofSeconds(2));
    CompletableFuture<Optional<List<OrderDTO>>> orders = callWithFallback(
        () → orderService.getOrders(userId), Duration.ofSeconds(3));
    CompletableFuture<Optional<List<NotifDTO>>> notifs = callWithFallback(
        () → notifService.getNotifs(userId), Duration.ofSeconds(2));

    CompletableFuture.allOf(user, orders, notifs).join();

    return DashboardDTO.builder()
        .user(user.join().orElse(UserDTO.anonymous()))
        .orders(orders.join().orElse(Collections.emptyList()))
        .notifications(notifs.join().orElse(Collections.emptyList()))
        .build();
}

private <T> CompletableFuture<Optional<T>> callWithFallback(
        Supplier<T> call, Duration timeout) {
    return CompletableFuture.supplyAsync(() → Optional.of(call.get()))
        .orTimeout(timeout.toMillis(), TimeUnit.MILLISECONDS)
        .exceptionally(ex → Optional.empty());
}
```

16.3 More Advanced Questions (Q51-Q80)

Q51. What are the key differences between `Stream.peek()` and `Stream.map()`?

> `peek()` takes a Consumer (returns void), is meant for side effects (debugging), and returns the same stream type. `map()` takes a Function (returns value), transforms each element, and can change the stream type. `peek()` may be optimized away by the JVM if it determines the operations have no observable effect.

Q52. How do you convert a stream operation to handle null elements safely?

```
// Filter nulls first
list.stream().filter(Objects::nonNull).map(String::toUpperCase);

// Or wrap with Optional
list.stream().map(Optional::ofNullable)
    .filter(Optional::isPresent).map(Optional::get);
```

Q53. How would you implement memoization using Java 8?

```
public class Memoizer {
    public static <T, R> Function<T, R> memoize(Function<T, R> fn) {
        Map<T, R> cache = new ConcurrentHashMap<>();
        return input -> cache.computeIfAbsent(input, fn);
    }
}
// Usage
Function<Integer, BigInteger> factorial = Memoizer.memoize(n ->
    IntStream.rangeClosed(1, n).mapToObj(BigInteger::valueOf)
        .reduce(BigInteger.ONE, BigInteger::multiply));
```

Q54. How do parallel streams handle stateful operations like sorted() and distinct()?

`sorted()` in parallel: each chunk is sorted independently, then results are merge-sorted during the combine phase (similar to merge sort). `distinct()` in parallel: uses ConcurrentHashMap internally to track seen elements across threads. Both operations add synchronization overhead that can reduce parallel performance gains.

Q55. How do you create a stream from a legacy Iterator or Enumeration?

```
// From Iterator
Iterator<String> iterator = legacyCollection.iterator();
Spliterator<String> spliterator = Spliterators.spliteratorUnknownSize(
    iterator, Spliterator.ORDERED);
Stream<String> stream = StreamSupport.stream(spliterator, false);

// From Enumeration
Enumeration<String> enumeration = legacyVector.elements();
Stream<String> stream = Collections.list(enumeration).stream();
```

Q56. What is short-circuit evaluation in Streams and which operations support it?

Short-circuit operations can terminate the pipeline without processing all elements. **Intermediate:** `limit(n)` stops after n elements. **Terminal:** `findFirst()`, `findAny()` stop after first match; `anyMatch()` stops at first true; `allMatch()` stops at first false; `noneMatch()` stops at first true. This is highly efficient for operations like searching in large datasets.

Q57. How does `Map.merge()` work with functional interfaces?

```
// merge(key, value, remappingFunction)
// If key absent → put(key, value)
// If key present → put(key, remapping.apply(oldValue, newValue))
Map<String, Integer> wordCount = new HashMap<>();
for (String word : words) {
    wordCount.merge(word, 1, Integer::sum);
}

// Other Map functional methods:
map.computeIfAbsent(key, k -> new ArrayList<>());
map.computeIfPresent(key, (k, v) -> v + 1);
map.compute(key, (k, v) -> v == null ? 1 : v + 1);
map.replaceAll((k, v) -> v.toUpperCase());
map.forEach((k, v) -> System.out.println(k + "=" + v));
```

Q58. Can you explain the concept of "effectively final" with respect to anonymous classes vs lambdas?

> Both anonymous classes and lambdas require captured local variables to be effectively final. However: anonymous classes CAN shadow variables from the enclosing scope (define a variable with the same name); lambdas CANNOT shadow them. This is because lambdas don't create a new scope—they share the enclosing method's scope.

Q59. How do you implement the Observer pattern using Java 8 built-in functional interfaces?

```
public class EventBus<T> {
    private final Map<String, List<Consumer<T>>> listeners =
        new ConcurrentHashMap<>();

    public void subscribe(String event, Consumer<T> handler) {
        listeners.computeIfAbsent(event, k -> new CopyOnWriteArrayList<>())
                 .add(handler);
    }

    public void publish(String event, T data) {
        listeners.getOrDefault(event, Collections.emptyList())
                 .forEach(handler -> handler.accept(data));
    }
}

// Usage
EventBus<Order> bus = new EventBus<>();
bus.subscribe("order.created", emailService::sendConfirmation);
bus.subscribe("order.created", inventoryService::reserve);
bus.subscribe("order.created", analyticsService::track);
```

Q60. How would you debug a complex stream pipeline?

> 1) **peek()** for logging intermediate values. 2) **Break pipeline** into smaller steps and store intermediate results. 3) **IDE debugging** (IntelliJ has Stream debugger that shows each step). 4) Use **stream().sequential()** for debugging parallel issues. 5) Extract lambdas to named methods for stack trace readability. 6) Use **logging** instead of peek in production. 7) Replace stream with **for-loop** temporarily to isolate the issue.

Q61. What happens if you call stream() on a Collection that is being modified concurrently?

> You'll get `ConcurrentModificationException` for fail-fast collections (ArrayList, HashMap). For concurrent collections (ConcurrentHashMap, CopyOnWriteArrayList), streams are designed to tolerate concurrent modifications and reflect a snapshot of the data. Always use concurrent collections or create a copy before streaming if the source is modified concurrently.

Q62. How do you implement backpressure with CompletableFuture?

> CompletableFuture doesn't have native backpressure (unlike reactive streams). Implement manually using: 1) **Semaphore** to limit concurrent operations. 2) **Bounded queue + Thread pool** with rejection policy. 3) **Batch processing** with controlled chunk sizes. For true backpressure, migrate to Project Reactor or RxJava.

```
Semaphore semaphore = new Semaphore(10); // Max 10 concurrent
List<CompletableFuture<Void>> futures = items.stream()
    .map(item -> CompletableFuture.runAsync(() -> {
        semaphore.acquireUninterruptibly();
        try { process(item); }
        finally { semaphore.release(); }
    })).collect(Collectors.toList());
CompletableFuture.allOf(futures.toArray(new CompletableFuture[0])).join();
```

Q63. How do primitive streams (IntStream, LongStream, DoubleStream) improve performance?

> They avoid autoboxing/unboxing overhead (int ↔ Integer). In a tight loop processing millions of elements, boxing creates millions of temporary wrapper objects, increasing GC pressure and cache misses. Primitive streams store values as raw primitives, provide specialized operations (sum, average, min, max), and have range methods (`IntStream.range()`). Always convert with `mapToInt()`, `mapToDouble()` when doing numeric operations.

Q64. What is the difference between `Stream.of(array)` and `Arrays.stream(array)`?

> For object arrays, they're equivalent. For primitive arrays:
> `Arrays.stream(int[])` → `IntStream` (correct, efficient)
> `Stream.of(int[])` → `Stream<int[]>` (wrong! creates a stream with one element: the array itself). Always use `Arrays.stream()` for primitive arrays.

Q65. When should you prefer `forEachOrdered()` over `forEach()`?

> Use `forEachOrdered()` in parallel streams when output order must match encounter order. `forEach()` in parallel has no order guarantee. In sequential streams, both behave identically. Example: writing to a file in order requires `forEachOrdered()`, but logging (where order doesn't matter) can use `forEach()`.

Q66-Q80: Quick-fire Questions

Q66: Can a functional interface extend another interface? **A:** Yes, but the resulting interface must still have exactly one abstract method.

Q67: Can a lambda expression throw checked exceptions? **A:** Only if the functional interface's abstract method declares them.

Q68: What does `Comparator.naturalOrder()` return? **A:** A comparator comparing Comparable objects in natural order.

Q69: What is `Comparator.nullsFirst()`? **A:** Wraps a comparator to handle null values, placing nulls before non-null.

Q70: Can streams be reused? **A:** No, a stream can only be consumed once. Use a Supplier of streams.

Q71: What is `Stream.concat()`? **A:** Creates a lazily concatenated stream from two streams.

Q72: How is `Optional.or()` different from `Optional.orElse()`? **A:** `or()` (Java 9) returns an Optional, `orElse()` returns the unwrapped value.

Q73: What does `Stream.generate()` do? **A:** Creates infinite stream using a Supplier.

Q74: Is Optional serializable? **A:** No, by design. Don't use it as a field type.

Q75: What is a `BiConsumer`? **A:** A Consumer that accepts two arguments. Used in Map.forEach().

Q76: How to get the current thread name in a parallel stream? **A:** `Thread.currentThread().getName()` in a peek/map.

Q77: What is `Collectors.collectingAndThen()`? **A:** Performs a final transformation on the collected result.

Q78: Can default methods be synchronized? **A:** No, the synchronized modifier is not allowed on default methods.

Q79: How many threads does the common ForkJoinPool use? **A:** `Runtime.getRuntime().availableProcessors() - 1`.

Q80: Can you override a default method in an implementing class? **A:** Yes, it works just like overriding any inherited method.

Chapter 17: Coding Exercises

20+ hands-on problems commonly asked in Java 8 interviews

17.1 Stream Basics

Exercise 1: Find duplicates in a list using Streams

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 2, 3, 5, 6, 5);

// Solution 1: Using groupingBy and filtering
List<Integer> duplicates = numbers.stream()
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() > 1)
    .map(Map.Entry::getKey)
    .collect(Collectors.toList());
// Result: [2, 3, 5]

// Solution 2: Using Set to track seen elements
Set<Integer> seen = new HashSet<>();
List<Integer> duplicates2 = numbers.stream()
    .filter(n -> !seen.add(n))  // add() returns false if already present
    .collect(Collectors.toList());
// Result: [2, 3, 5]

// Solution 3: Using Collections.frequency
List<Integer> duplicates3 = numbers.stream()
    .filter(n -> Collections.frequency(numbers, n) > 1)
    .distinct()
    .collect(Collectors.toList());
```

Exercise 2: Group employees by department and find highest salary in each

```
List<Employee> employees = Arrays.asList(
    new Employee("Alice", "Engineering", 95000),
    new Employee("Bob", "Engineering", 110000),
    new Employee("Charlie", "Marketing", 75000),
    new Employee("Diana", "Marketing", 82000),
    new Employee("Eve", "HR", 68000)
);

Map<String, Optional<Employee>> highestByDept = employees.stream()
    .collect(Collectors.groupingBy(
        Employee::getDepartment,
        Collectors.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));
// {Engineering=Optional[Bob(110000)], Marketing=Optional[Diana(82000)], HR=Opt

// Without Optional wrapper:
Map<String, Employee> highestByDept2 = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        (e1, e2) -> e1.getSalary() > e2.getSalary() ? e1 : e2
    ));
```

Exercise 3: Sort a list of custom objects by multiple fields

```
List<Employee> sorted = employees.stream()
    .sorted(Comparator
        .comparing(Employee::getDepartment)                    // Primary sort
        .thenComparing(Employee::getSalary, Comparator.reverseOrder())  // Seco
        .thenComparing(Employee::getName))                     // Tertiary
    .collect(Collectors.toList());

// With null handling
List<Employee> nullSafeSorted = employees.stream()
    .sorted(Comparator.comparing(
        Employee::getManager,
        Comparator.nullsLast(Comparator.naturalOrder())))
    .collect(Collectors.toList());
```

Exercise 4: Convert a List to Map (handling duplicate keys)

```
List<Employee> employees = getEmployees(); // May have same department

// Simple: ID as key (assuming unique IDs)
Map<Long, Employee> byId = employees.stream()
    .collect(Collectors.toMap(Employee::getId, Function.identity()));

// With duplicate key handling: keep the one with higher salary
Map<String, Employee> topByDept = employees.stream()
    .collect(Collectors.toMap(
        Employee::getDepartment,
        Function.identity(),
        BinaryOperator.maxBy(Comparator.comparingDouble(Employee::getSalary))
    ));

// Collecting to LinkedHashMap (maintaining insertion order)
LinkedHashMap<Long, String> orderedMap = employees.stream()
    .sorted(Comparator.comparing(Employee::getName))
    .collect(Collectors.toMap(
        Employee::getId,
        Employee::getName,
        (v1, v2) -> v1,
        LinkedHashMap::new
    ));
```

17.2 Intermediate Problems

Exercise 5: Flatten nested lists and find unique elements

```
List<List<String>> nested = Arrays.asList(
    Arrays.asList("Java", "Python", "C++"),
    Arrays.asList("Python", "JavaScript", "Go"),
    Arrays.asList("Java", "Rust", "Go")
);

// Flatten and get unique languages
List<String> uniqueLanguages = nested.stream()
    .flatMap(Collection::stream)
    .distinct()
    .sorted()
    .collect(Collectors.toList());
// Result: [C++, Go, Java, JavaScript, Python, Rust]

// Count occurrences
Map<String, Long> languageCount = nested.stream()
    .flatMap(Collection::stream)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
// {Java=2, Python=2, C++=1, JavaScript=1, Go=2, Rust=1}
```

Exercise 6: Find Top N elements

```
// Top 3 highest paid employees
List<Employee> top3 = employees.stream()
    .sorted(Comparator.comparingDouble(Employee::getSalary).reversed())
    .limit(3)
    .collect(Collectors.toList());

// Top 3 most common words in a text
String text = "the quick brown fox jumps over the lazy dog the fox";
Map<String, Long> top3Words = Arrays.stream(text.split("\\s+"))
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(3)
    .collect(Collectors.toMap(
        Map.Entry::getKey, Map.Entry::getValue,
        (e1, e2) -> e1, LinkedHashMap::new));
// {the=3, fox=2, quick=1}
```

Exercise 7: String Processing – Find longest string, character frequency

```
List<String> words = Arrays.asList("Java", "Stream", "Lambda", "CompletableFutu

// Find the longest string
String longest = words.stream()
    .max(Comparator.comparingInt(String::length))
    .orElse("");
// "CompletableFuture"

// Character frequency in a string
Map<Character, Long> charFreq = "programming".chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
// {p=1, r=2, o=1, g=2, a=1, m=2, i=1, n=1}

// Find first non-repeated character
Character firstNonRepeated = "aabcbdeffc".chars()
    .mapToObj(c -> (char) c)
    .collect(Collectors.groupingBy(Function.identity(),
             LinkedHashMap::new, Collectors.counting()))
    .entrySet().stream()
    .filter(e -> e.getValue() == 1)
    .map(Map.Entry::getKey)
    .findFirst()
    .orElse(null);
// 'd'
```

Exercise 8: Partition employees into salary ranges

```
// Partition into above/below average
double avgSalary = employees.stream()
    .mapToDouble(Employee::getSalary)
    .average()
    .orElse(0);

Map<Boolean, List<Employee>> partitioned = employees.stream()
    .collect(Collectors.partitioningBy(e -> e.getSalary() > avgSalary));

System.out.println("Above average: " + partitioned.get(true));
System.out.println("Below average: " + partitioned.get(false));

// Custom salary ranges
Map<String, List<Employee>> ranges = employees.stream()
    .collect(Collectors.groupingBy(e -> {
        if (e.getSalary() < 50000) return "Low";
        if (e.getSalary() < 100000) return "Medium";
        return "High";
    }));
```

17.3 Advanced Problems

Exercise 9: Implement pagination with streams

```
public <T> List<T> getPage(List<T> data, int page, int size) {
    return data.stream()
        .skip((long) (page - 1) * size)
        .limit(size)
        .collect(Collectors.toList());
}

// Usage
List<Employee> page2 = getPage(allEmployees, 2, 10); // Page 2, 10 items/page

// With sorting
public Page<Employee> getEmployees(int page, int size, String sortBy) {
    List<Employee> all = employeeRepository.findAll();
    List<Employee> sorted = all.stream()
        .sorted(Comparator.comparing(e -> getField(e, sortBy)))
        .collect(Collectors.toList());
    List<Employee> pageData = sorted.stream()
        .skip((long) (page - 1) * size)
        .limit(size)
        .collect(Collectors.toList());
    return new Page<>(pageData, all.size(), page, size);
}
```

Exercise 10: Build an SQL-like IN clause from a list

```
List<Long> ids = Arrays.asList(101L, 102L, 103L, 104L);

String inClause = ids.stream()
    .map(String::valueOf)
    .collect(Collectors.joining(", ", "SELECT * FROM orders WHERE id IN (", ")"
// "SELECT * FROM orders WHERE id IN (101, 102, 103, 104)"

// Batched IN clauses (for databases with limit on IN clause size)
int batchSize = 1000;
List<List<Long>> batches = IntStream.range(0, (ids.size() + batchSize - 1) / ba
    .mapToObj(i -> ids.subList(i * batchSize,
              Math.min((i + 1) * batchSize, ids.size())))
    .collect(Collectors.toList());
```

Exercise 11: Calculate running total/cumulative sum

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5);

// Running total
AtomicInteger runningTotal = new AtomicInteger(0);
List<Integer> cumulativeSum = numbers.stream()
    .map(n -> runningTotal.addAndGet(n))
    .collect(Collectors.toList());
// [1, 3, 6, 10, 15]

// Using IntStream.reduce (pure functional approach)
int[] cumSum = new int[numbers.size()];
IntStream.range(0, numbers.size())
    .forEach(i -> cumSum[i] = (i == 0 ? 0 : cumSum[i-1]) + numbers.get(i));
// [1, 3, 6, 10, 15]
```

Exercise 12: Find second highest salary / Nth highest

```
// Second highest salary
Optional<Double> secondHighest = employees.stream()
    .map(Employee::getSalary)
    .distinct()
    .sorted(Comparator.reverseOrder())
    .skip(1)
    .findFirst();

// Nth highest (generic)
public Optional<Double> getNthHighestSalary(List<Employee> employees, int n) {
    return employees.stream()
        .map(Employee::getSalary)
        .distinct()
        .sorted(Comparator.reverseOrder())
        .skip(n - 1)
        .findFirst();
}

// Employees with second highest salary (may be multiple)
double secondMax = employees.stream()
    .mapToDouble(Employee::getSalary)
    .distinct()
    .sorted()
    .skip(employees.stream().mapToDouble(Employee::getSalary)
          .distinct().count() - 2)
    .findFirst().orElse(0);
List<Employee> secondHighestEmps = employees.stream()
    .filter(e -> e.getSalary() == secondMax)
    .collect(Collectors.toList());
```

Exercise 13: Merge two maps using streams

```
Map<String, Integer> map1 = Map.of("a", 1, "b", 2, "c", 3);
Map<String, Integer> map2 = Map.of("b", 20, "c", 30, "d", 40);

// Merge with sum for duplicate keys
Map<String, Integer> merged = Stream.concat(
        map1.entrySet().stream(),
        map2.entrySet().stream())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        Integer::sum
    ));
// {a=1, b=22, c=33, d=40}

// Merge keeping latest value
Map<String, Integer> mergedLatest = Stream.of(map1, map2)
    .flatMap(m -> m.entrySet().stream())
    .collect(Collectors.toMap(
        Map.Entry::getKey,
        Map.Entry::getValue,
        (v1, v2) -> v2  // Keep value from map2
    ));
```

Exercise 14: FizzBuzz with Streams

```
List<String> fizzBuzz = IntStream.rangeClosed(1, 100)
    .mapToObj(n -> {
        if (n % 15 == 0) return "FizzBuzz";
        if (n % 3 == 0) return "Fizz";
        if (n % 5 == 0) return "Buzz";
        return String.valueOf(n);
    })
    .collect(Collectors.toList());
```

Exercise 15: Fibonacci sequence with streams

```
// Generate first 20 Fibonacci numbers
Stream.iterate(new long[]{0, 1}, f -> new long[]{f[1], f[0] + f[1]})
    .limit(20)
    .map(f -> f[0])
    .forEach(System.out::println);
// 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, ...
```

Exercise 16: Find all pairs that sum to a target

```
List<Integer> numbers = Arrays.asList(1, 2, 3, 4, 5, 6, 7, 8, 9);
int target = 10;

Set<Integer> numSet = new HashSet<>(numbers);
List<int[]> pairs = numbers.stream()
    .filter(n -> n < target - n)  // Avoid duplicates
    .filter(n -> numSet.contains(target - n))
    .map(n -> new int[]{n, target - n})
    .collect(Collectors.toList());
// [[1,9], [2,8], [3,7], [4,6]]
```

Exercise 17: Transform CSV data using streams

```
String csv = "Name,Age,City\nAlice,30,New York\nBob,25,London\nCharlie,35,Tokyo

List<Map<String, String>> records = Arrays.stream(csv.split("\n"))
    .skip(1) // Skip header
    .map(line -> {
        String[] fields = line.split(",");
        Map<String, String> record = new LinkedHashMap<>();
        record.put("name", fields[0]);
        record.put("age", fields[1]);
        record.put("city", fields[2]);
        return record;
    })
    .collect(Collectors.toList());

// Filter and transform
List<String> youngPeopleInCities = records.stream()
    .filter(r -> Integer.parseInt(r.get("age")) < 30)
    .map(r -> r.get("name") + " lives in " + r.get("city"))
    .collect(Collectors.toList());
```

Exercise 18: Parallel processing simulation

```
// Simulate API calls with timeouts
List<String> urls = List.of(
    "https://api.service1.com/data",
    "https://api.service2.com/data",
    "https://api.service3.com/data"
);

ExecutorService executor = Executors.newFixedThreadPool(3);
List<CompletableFuture<ApiResponse>> futures = urls.stream()
    .map(url -> CompletableFuture.supplyAsync(() -> callApi(url), executor)
        .orTimeout(5, TimeUnit.SECONDS)
        .exceptionally(ex -> ApiResponse.error(url, ex.getMessage())))
    .collect(Collectors.toList());

List<ApiResponse> responses = futures.stream()
    .map(CompletableFuture::join)
    .collect(Collectors.toList());

Map<Boolean, List<ApiResponse>> partitioned = responses.stream()
    .collect(Collectors.partitioningBy(ApiResponse::isSuccess));

System.out.println("Successful: " + partitioned.get(true).size());
System.out.println("Failed: " + partitioned.get(false).size());
executor.shutdown();
```

Exercise 19: Word frequency counter from file

```
// Count word frequency from a file
Map<String, Long> wordFrequency = Files.lines(Paths.get("document.txt"))
    .flatMap(line -> Arrays.stream(line.toLowerCase().split("[^a-zA-Z]+")))
    .filter(word -> !word.isEmpty())
    .collect(Collectors.groupingBy(Function.identity(), Collectors.counting()))
    .entrySet().stream()
    .sorted(Map.Entry.<String, Long>comparingByValue().reversed())
    .limit(20)  // Top 20 words
    .collect(Collectors.toMap(
        Map.Entry::getKey, Map.Entry::getValue,
        (e1, e2) -> e1, LinkedHashMap::new
    ));
```

Exercise 20: Matrix operations with streams

```
int[][] matrix = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};

// Flatten matrix
int[] flat = Arrays.stream(matrix)
    .flatMapToInt(Arrays::stream)
    .toArray();
// [1, 2, 3, 4, 5, 6, 7, 8, 9]

// Sum of all elements
int total = Arrays.stream(matrix)
    .flatMapToInt(Arrays::stream)
    .sum();
// 45

// Transpose matrix
int rows = matrix.length;
int cols = matrix[0].length;
int[][] transposed = IntStream.range(0, cols)
    .mapToObj(j -> IntStream.range(0, rows)
        .map(i -> matrix[i][j])
        .toArray())
    .toArray(int[][]::new);
```

🎯 Interview Coding Tips

- Always clarify requirements: handle nulls? handle duplicates? sort order?
- Start with the simplest stream solution, then optimize if needed
- Know when NOT to use streams — simple loops may be clearer for mutations
- Practice converting between stream and loop solutions
- Mention performance implications of your approach
- Discuss alternative approaches (e.g., collect vs reduce)